M06 — Comunicación entre microservicios con ASP.NET Core 8 · Teoría

Autor del curso: JAIME GALLO

Cómo estudiar el módulo

Los tres puntos conservan diez bloques cada uno y desarrollan HTTP, eventos y criterios de decisión. Cada bloque añade una ampliación aplicada a AceriaApp con decisión, ejemplo, límite y comprobación. El material incluye un mapa general y nueve diagramas de detalle, todos disponibles en SVG vectorial y PNG. Las explicaciones **Línea N** corresponden al número físico del ejemplo inmediatamente anterior, incluidas las líneas vacías; estas no necesitan explicación. Cada explicación ocupa una viñeta separada.

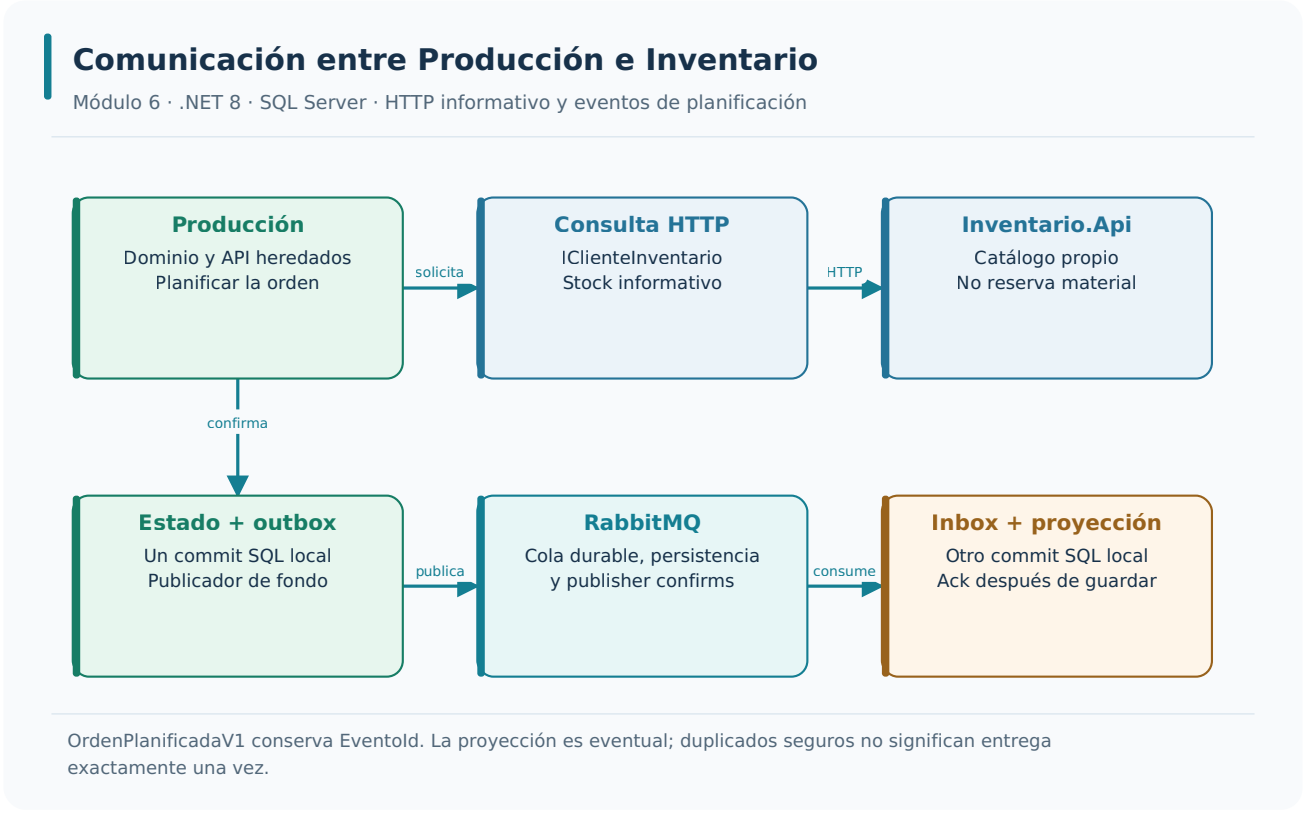
Punto de partida. Se continúa desde `M05_DDD_CQRS/PROYECTO/5.5-dominio-aceria`. La API heredada mantiene el contexto de Producción, sus entidades, value objects, casos de uso, commands, queries y persistencia SQL Server. El módulo añade `Inventario.Api` como proceso independiente. La frontera de red obliga a diseñar errores, cancelación y contratos; no autoriza a compartir tablas ni entidades entre contextos.

Recorrido funcional. El punto 6.1 consulta disponibilidad informativa mediante el puerto `IClienteInventario` y un cliente HTTP tipado. El punto 6.2 comunica `OrdenPlanificadaV1` con outbox SQL unido al cambio de estado de Producción, RabbitMQ y un consumidor con inbox SQL que registra una proyección de planificación en Inventario. El punto 6.3 combina consulta HTTP, planificación local y outbox, respuesta `202 Accepted` y propagación en segundo plano. La lectura de la proyección es eventual. Ninguno de esos pasos reserva ni descuenta stock.

Convenciones de ejemplos. Los fragmentos se estudian de forma independiente y muestran las partes relevantes; las prácticas contienen archivos completos, nombres y rutas de ejecución. Los ejemplos C# presuponen los namespaces correspondientes de `System.Net`, `System.Net.Http.Json`, `System.Text.Json`, ASP.NET Core y, donde se indique, `RabbitMQ.Client`. DTOs y interfaces auxiliares de los ejemplos son contratos didácticos; no se copian todos juntos. El stack del recorrido es .NET 8, EF Core 8 y SQL Server real, con RabbitMQ.Client 7.1.2 para los ejemplos de mensajería. El tiempo de estudio y laboratorio depende de las instalaciones, diagnóstico y pruebas: las prácticas ampliadas no se prometen en veintisiete minutos.

Operación de los trabajadores. La publicación y el consumo se habilitan por configuración. El consumidor usa `BasicGetAsync` con sondeo y ack manual para que el laboratorio sea fácil de observar; no se presenta como diseño de alto rendimiento. Los errores de formato o versión se dirigen a DLQ y los fallos técnicos se reencolan. La proyección usa unicidad por EventoId y OrdenId y una huella del payload para detectar reutilización incompatible de identidad.

Alcance y evidencia. Reserva, liberación, facturación, Saga, coordinación distribuida y Event Sourcing sirven para analizar alternativas cuando se nombran como ejemplos conceptuales. No se presentan como capacidades implementadas. Una lista de conceptos o un test de coherencia no sustituye a pruebas de comportamiento, persistencia, mensajería y recorrido HTTP. Los mecanismos de confiabilidad tienen supuestos y límites: no se promete entrega ni efecto global «exactamente una vez».



Interpretación del mapa: Producción confirma estado y outbox en una transacción local. La consulta HTTP es informativa. Publicador, broker y consumidor propagan el hecho a otra transacción local, con posible retraso y reentrega.

Punto 6.1 — Comunicación síncrona mediante HTTP

Módulo 6: Comunicación entre microservicios **Objetivos de aprendizaje:**

- Comprender qué es la comunicación síncrona entre microservicios.
- Identificar el papel de HTTP y de los clientes tipados en la comunicación síncrona.
- Comprender la relación entre comunicación síncrona y acoplamiento temporal.
- Comprender los riesgos de la comunicación síncrona en sistemas distribuidos.
- Aplicar los conceptos al recorrido real de Producción e Inventario y verificar su comportamiento.

Desarrollo teórico

Bloque 1 — Definición de comunicación síncrona

La comunicación síncrona es un estilo de interacción en el que un servicio envía una petición a otro servicio y espera la respuesta antes de continuar con su propia ejecución. El flujo que necesita el resultado espera hasta recibir una respuesta, una cancelación o un fallo. Esto no significa que un hilo permanezca bloqueado: `await` permite liberar el hilo durante la espera de E/S. La comunicación síncrona a nivel de interacción puede implementarse con APIs asíncronas de .NET. HTTP/1.1 y HTTP/2 utilizan TCP; HTTP/3 utiliza QUIC sobre UDP. La petición identifica el endpoint del servicio destino y la respuesta se correlaciona con ella. El servicio origen y el servicio destino deben estar disponibles al mismo tiempo para que la comunicación se complete.

```
Comunicación síncrona:

Servicio A → petición HTTP → Servicio B.
Servicio A ← respuesta HTTP ← Servicio B.
```

- **Línea 1:** `Comunicación síncrona:` → identifica el tema del ejemplo.
- **Línea 3:** `Servicio A → petición HTTP → Servicio B.` → el servicio A envía la petición.
- **Línea 4:** `Servicio A ← respuesta HTTP ← Servicio B.` → el servicio A recibe la respuesta.

La comunicación síncrona se caracteriza por tres propiedades. La primera es que el emisor espera la respuesta. La segunda es que el receptor debe estar disponible en el momento de la petición. La tercera es que la comunicación tiene un tiempo de espera máximo. Las tres propiedades permiten que la interacción sea simple pero introducen acoplamiento temporal.

La aplicación de la comunicación síncrona tiene dos consecuencias prácticas. La primera es que el flujo de datos es directo y fácil de seguir. La segunda es que el fallo del servicio destino afecta al servicio origen. La combinación de ambas permite que la comunicación sea simple pero también frágil ante fallos.

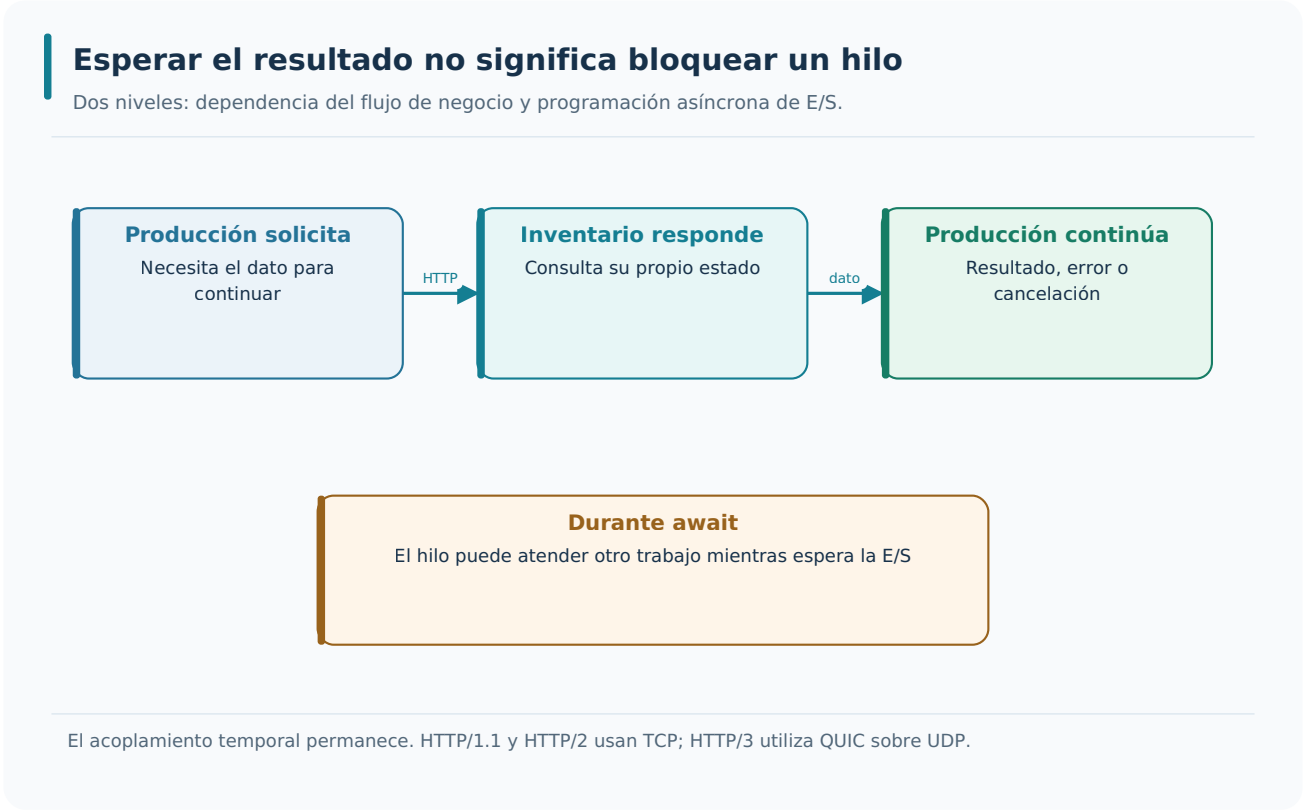
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Distingue dos niveles de asincronía. En el nivel del programa, un método devuelve Task y libera el hilo mientras espera E/S. En el nivel del negocio, Producción sigue necesitando el resultado de Inventario para decidir su siguiente paso. Usar async no rompe esa dependencia temporal.

Ejemplo concreto en AceriaApp. Cuando un planificador solicita información de CHAPA-3MM, el cliente HTTP espera los datos. Mientras llega la respuesta, ASP.NET Core puede atender otras solicitudes. La continuación del caso de uso se ejecuta después del await; no significa que toda la API se detenga.

Riesgo, alternativa y límite. No sustituyas await por .Result o .Wait() para hacer la lectura más «síncrona». Esas llamadas bloquean un hilo y pueden perjudicar la capacidad del servidor. Tampoco lances una Task sin observarla: perderías errores y la relación entre la solicitud y su resultado.

Comprobación técnica y aprendizaje. Introduce un retraso controlado en Inventario y observa dos hechos distintos: aumenta la latencia de la operación dependiente y otras rutas independientes pueden seguir respondiendo. Después cancela la solicitud y comprueba que el token llega al cliente. El experimento explica dependencia temporal sin confundirla con bloqueo de hilos.



Interpretación del gráfico: La interacción espera el resultado remoto, mientras await permite liberar el hilo durante la E/S. La figura se distribuye en SVG vectorial y PNG.

Bloque 2 — HTTP como protocolo de comunicación síncrona

HTTP es el protocolo más usado para la comunicación síncrona entre microservicios. HTTP define una semántica de petición-respuesta independiente de su versión de transporte: HTTP/1.1 y HTTP/2 usan TCP; HTTP/3 usa QUIC sobre UDP. Cada petición contiene un método, una URL, cabeceras y un cuerpo opcional. Cada respuesta contiene un código de estado, cabeceras y un cuerpo opcional. HTTP es ubicuo, está soportado por todas las plataformas y se integra con herramientas de monitorización. HTTP/2 y HTTP/3 permiten multiplexar intercambios y comprimen cabeceras con mecanismos diferentes. Esto puede mejorar el uso de la conexión, pero no elimina la latencia de la aplicación ni garantiza mejor rendimiento en cualquier escenario.

```
using var response = await httpClient.GetAsync("api/v1/materiales/CHAPA-3MM", ct);
response.EnsureSuccessStatusCode();
var material = await response.Content.ReadFromJsonAsync<MaterialDto>(cancellationToken: ct);
if (material is null) throw new InvalidDataException("Respuesta JSON vacía.");
```

- **Línea 1:** `using var response = await httpClient.GetAsync("api/v1/materiales/CHAPA-3MM", ct);` → envía GET con cancelación y dispone la respuesta al salir del ámbito; el cliente ya está inyectado.
- **Línea 2:** `response.EnsureSuccessStatusCode();` → rechaza estados distintos de 2xx; el siguiente bloque distingue específicamente 404.
- **Línea 3:** `var material = await response.Content.ReadFromJsonAsync<MaterialDto>(cancellationToken: ct);` → deserializa el contrato y propaga el token también durante la lectura.
- **Línea 4:** `if (material is null) throw new InvalidDataException("Respuesta JSON vacía.");` → rechaza un cuerpo que no contiene el DTO esperado, en vez de inventar un resultado válido.

HTTP como protocolo se caracteriza por tres propiedades. La primera es que es un protocolo de petición-respuesta. La segunda es que está soportado por todas las plataformas. La tercera es que se integra con herramientas de monitorización. Las tres propiedades permiten que la comunicación síncrona sea universal y observable.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El contrato HTTP incluye ruta, método, códigos de estado, tipo de contenido y cuerpo. Una conexión exitosa solo indica que hubo intercambio de bytes; una respuesta 200 con un cuerpo incompatible sigue incumpliendo el contrato que necesita el consumidor.

Ejemplo concreto en AceriaApp. Para una consulta de material, 200 con DTO representa un material conocido, 404 representa que ese recurso no existe y 500 o 503 representan un fallo técnico. Esos resultados no deben reducirse todos a false: de lo contrario una caída de Inventario se presentaría como falta real de stock.

Riesgo, alternativa y límite. HTTP/3 usa QUIC y no TCP. La diferencia de transporte no modifica la necesidad de validar el resultado ni convierte una lectura de stock en una reserva. TLS, autenticación y permisos también deben diseñarse según el entorno; una URL local del laboratorio no es una política de despliegue.

Comprobación técnica y aprendizaje. Comprueba por separado respuestas 200, 404, 500 y JSON inválido. Asegura que HttpResponseMessage se dispone aun cuando la deserialización lance una excepción. El cliente puede reutilizar conexiones administradas por la factoría sin conservar indefinidamente los objetos de respuesta.

Bloque 3 — Clientes HTTP tipados

Un cliente HTTP tipado es una clase que encapsula las llamadas a un servicio específico. El cliente se registra en el contenedor de dependencias con `AddHttpClient`. `IHttpClientFactory` crea clientes configurados y administra la reutilización y renovación de los manejadores HTTP subyacentes. No conviene capturar un cliente tipado transitorio dentro de un singleton de larga duración. El cliente expone métodos específicos del servicio: `ObtenerMaterial`, `CrearOrden`, `CancelarOrden`. Los métodos devuelven contratos de aplicación, normalmente DTOs adaptados a la necesidad local; no deben importar las entidades persistentes del servicio remoto. El cliente tipado abstrae los detalles de HTTP y centraliza la configuración.

```
public interface IClienteInventario
{
    Task<MaterialDto?> ObtenerMaterialAsync(string nombre, CancellationToken ct);
    Task<bool> MaterialDisponibleAsync(string nombre, int cantidad, CancellationToken ct);
}

public sealed class ClienteInventarioHttp(HttpClient httpClient) : IClienteInventario
{
    public async Task<MaterialDto?> ObtenerMaterialAsync(string nombre, CancellationToken ct)
    {
        var ruta = $"api/v1/materiales/{Uri.EscapeDataString(nombre)}";
        using var response = await httpClient.GetAsync(ruta, ct);
        if (response.StatusCode == HttpStatusCode.NotFound) return null;
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<MaterialDto>(cancellationToken: ct)
            ?? throw new InvalidDataException("El inventario devolvió un cuerpo vacío.");
    }

    public async Task<bool> MaterialDisponibleAsync(string nombre, int cantidad, CancellationToken
ct)
    {
        if (cantidad <= 0) throw new ArgumentOutOfRangeException(nameof(cantidad));
        var material = await ObtenerMaterialAsync(nombre, ct);
        return material is not null && material.Stock >= cantidad;
    }
}
```

- **Línea 1:** `public interface IClienteInventario` → define el puerto que usa la aplicación; el dominio no conoce HttpClient.
- **Línea 2:** `{` → abre el bloque de la declaración anterior.
- **Línea 3:** `Task<MaterialDto?> ObtenerMaterialAsync(string nombre, CancellationToken ct);` → distingue material inexistente con null y admite cancelación.
- **Línea 4:** `Task<bool> MaterialDisponibleAsync(string nombre, int cantidad, CancellationToken ct);` → expresa una consulta informativa de disponibilidad.
- **Línea 5:** `}` → cierra el bloque y, cuando corresponde, la instrucción.
- **Línea 7:** `public sealed class ClienteInventarioHttp(HttpClient httpClient) : IClienteInventario` → implementa el puerto mediante un cliente tipado inyectado; usa constructor primario de C# 12.
- **Línea 8:** `{` → abre el bloque de la declaración anterior.
- **Línea 9:** `public async Task<MaterialDto?> ObtenerMaterialAsync(string nombre, CancellationToken ct)` → declara la consulta HTTP asíncrona.
- **Línea 10:** `{` → abre el bloque de la declaración anterior.
- **Línea 11:** `var ruta = $"api/v1/materiales/{Uri.EscapeDataString(nombre)}";` → codifica el nombre como segmento de ruta para no alterar la URL.
- **Línea 12:** `using var response = await httpClient.GetAsync(ruta, ct);` → envía la solicitud con cancelación y garantiza la disposición de la respuesta.
- **Línea 13:** `if (response.StatusCode == HttpStatusCode.NotFound) return null;` → solo 404 se traduce en ausencia del recurso.
- **Línea 14:** `response.EnsureSuccessStatusCode();` → propaga como error técnico un 500, 503 u otro estado no satisfactorio.
- **Línea 15:** `return await response.Content.ReadFromJsonAsync<MaterialDto>(cancellationToken: ct)` → deserializa JSON con el token de cancelación.

- **Línea 16:** `?? throw new InvalidDataException("El inventario devolvió un cuerpo vacío.");` → considera inválida una respuesta exitosa sin el objeto esperado.
- **Línea 17:** `}` → cierra el bloque y, cuando corresponde, la instrucción.
- **Línea 19:** `public async Task<bool> MaterialDisponibleAsync(string nombre, int cantidad, CancellationToken ct)` → declara la comprobación informativa.
- **Línea 20:** `{` → abre el bloque de la declaración anterior.
- **Línea 21:** `if (cantidad <= 0) throw new ArgumentOutOfRangeException(nameof(cantidad));` → rechaza cantidades no positivas antes de consultar.
- **Línea 22:** `var material = await ObtenerMaterialAsync(nombre, ct);` → consulta el stock observado en ese instante.
- **Línea 23:** `return material is not null && material.Stock >= cantidad;` → compara el dato recibido; no reserva stock ni protege frente a otras operaciones.
- **Línea 24:** `}` → cierra el bloque y, cuando corresponde, la instrucción.
- **Línea 25:** `}` → cierra el bloque y, cuando corresponde, la instrucción.

El cliente tipado se caracteriza por tres propiedades. La primera es que encapsula las llamadas al servicio destino. La segunda es que expone métodos específicos del servicio. La tercera es que centraliza la configuración. Las tres propiedades permiten que la comunicación síncrona sea mantenible y testeable.

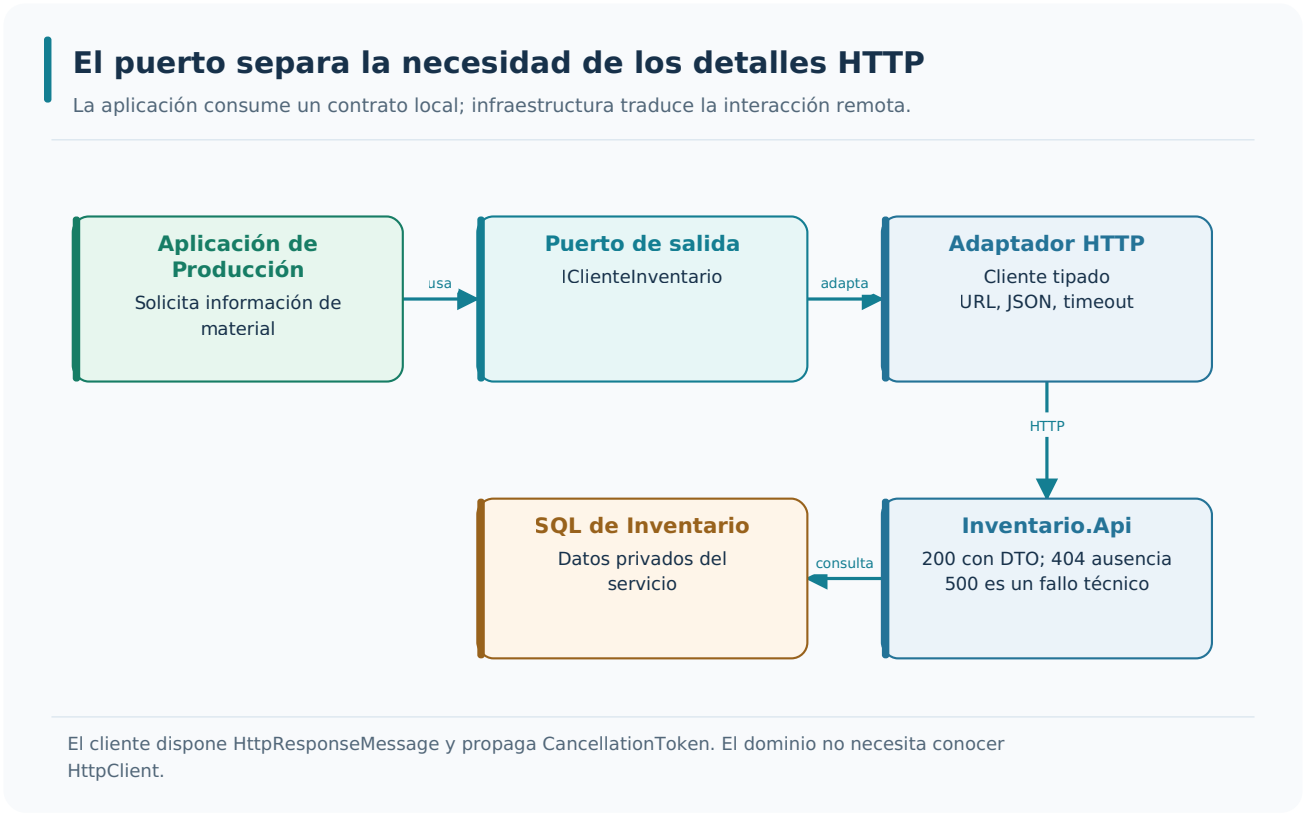
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El puerto IClienteInventario expresa lo que necesita Producción, mientras el adaptador sabe de URL, códigos HTTP y JSON. Esta separación permite probar el caso de uso con un doble del puerto y probar el mapeo del adaptador con respuestas HTTP controladas.

Ejemplo concreto en AceriaApp. ObtenerMaterialAsync puede devolver null para un 404 definido por contrato. MaterialDisponibleAsync compara la cantidad con el stock de ese instante. El nombre y la documentación deben hacer visible que se trata de una observación; no existe una sección crítica distribuida entre leer y planificar.

Riesgo, alternativa y límite. No exportes una entidad EF de Inventario para evitar escribir un DTO. Eso acoplaría el contrato a tablas, navegaciones y decisiones internas. El consumidor puede traducir el DTO remoto a una representación local, y los errores técnicos deben seguir siendo distinguibles de un resultado de negocio negativo.

Comprobación técnica y aprendizaje. Prueba un nombre que contiene espacios o caracteres reservados, una cantidad no positiva, una cancelación y un error 500. Comprueba que el escape de la ruta evita cambiar su estructura. Estas pruebas revisan comportamiento y contrato, no solo que exista una interfaz con el nombre esperado.



Interpretación del gráfico: La aplicación depende de un puerto; el adaptador traduce HTTP y la base de Inventario sigue siendo privada. La figura se distribuye en SVG vectorial y PNG.

Bloque 4 — Configuración del cliente HTTP tipado

La configuración del cliente HTTP tipado se realiza en el contenedor de dependencias con `AddHttpClient`. El método permite configurar la URL base, las cabeceras por defecto, los tiempos de espera y los manejadores de mensajes. La configuración se realiza una vez y se aplica a todas las llamadas del cliente. La configuración centralizada evita la duplicación de código.

```
builder.Services.AddHttpClient<IClienteInventario, ClienteInventarioHttp>(client =>
{
    client.BaseAddress = new Uri(builder.Configuration["Inventario:BaseUrl"]!);
    client.Timeout = TimeSpan.FromSeconds(5);
    client.DefaultRequestHeaders.Accept.ParseAdd("application/json");
});
```

- **Línea 1:** `builder.Services.AddHttpClient<IClienteInventario, ClienteInventarioHttp>(client =>` → registra el adaptador tipado para el puerto de aplicación.
- **Línea 2:** `{` → abre la configuración central del cliente.
- **Línea 3:** `client.BaseAddress = new Uri(builder.Configuration["Inventario:BaseUrl"]!);` → lee la URL del entorno; la configuración debe validarse al arrancar.
- **Línea 4:** `client.Timeout = TimeSpan.FromSeconds(5);` → limita la espera de una llamada a cinco segundos en este ejemplo.
- **Línea 5:** `client.DefaultRequestHeaders.Accept.ParseAdd("application/json");` → solicita una representación JSON mediante Accept.
- **Línea 6:** `});` → cierra el registro de dependencias.

La configuración del cliente tipado tiene dos consecuencias prácticas. La primera es que la configuración se centraliza en un único punto. La segunda es que la factoría administra los manejadores subyacentes y sus conexiones. La combinación de ambas permite que la comunicación síncrona sea robusta y mantenible.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Centralizar configuración permite cambiar el destino por entorno sin recompilar el dominio. La configuración debe validar que la URL existe, es absoluta y usa un esquema admitido. Un operador debería descubrir un error de configuración al arrancar, no tras la primera petición de un usuario.

Ejemplo concreto en AceriaApp. El laboratorio ejecuta Producción e Inventario como procesos distintos. Sus direcciones base deben corresponder a los puertos elegidos en las prácticas. El cliente añade rutas relativas; una barra inicial puede sustituir el prefijo de una URL base con subruta y cambiar el recurso solicitado.

Riesgo, alternativa y límite. La factoría administra manejadores y conexiones, no una garantía de disponibilidad. El timeout de cinco segundos es una decisión de ejemplo que debe coordinarse con el presupuesto del endpoint y con la política de reintentos. Tres intentos de cinco segundos no caben en una solicitud que promete terminar en seis.

Comprobación técnica y aprendizaje. Arranca con una URL incorrecta y revisa el mensaje de diagnóstico; luego usa la dirección válida y comprueba la consulta real. Revisa la vida del consumidor del cliente tipado: un singleton que lo captura puede impedir la renovación esperada de sus dependencias.

Bloque 5 — Acoplamiento temporal

La comunicación síncrona introduce acoplamiento temporal entre los servicios. El servicio origen debe esperar la respuesta del servicio destino. Si el servicio destino no está disponible, el servicio origen no puede completar su operación. El acoplamiento temporal es aceptable cuando la operación requiere la respuesta del destino para continuar. El acoplamiento temporal es problemático cuando la operación puede completarse sin la respuesta.

```
Acoplamiento temporal:

Servicio A espera → Servicio B responde → Servicio A continúa.
Servicio A espera → Servicio B no responde → Servicio A falla.
```

- **Línea 1:** `Acoplamiento temporal:` → identifica el tema del ejemplo.
- **Línea 3:** `Servicio A espera → Servicio B responde → Servicio A continúa.` → caso exitoso.
- **Línea 4:** `Servicio A espera → Servicio B no responde → Servicio A falla.` → caso de fallo.

El acoplamiento temporal tiene dos consecuencias prácticas. La primera es que la disponibilidad del servicio destino afecta al servicio origen. La segunda es que la latencia del servicio destino afecta a la latencia del servicio origen. La combinación de ambas permite que la comunicación síncrona sea directa pero también frágil.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Una cadena síncrona suma dependencias de tiempo y disponibilidad. Si A necesita B y B necesita C, el usuario de A observa retrasos y fallos de toda la cadena. El diseño debe justificar cada dependencia antes de añadir políticas que solo intentan amortiguarla.

Ejemplo concreto en AceriaApp. Producción consulta Inventario para informar una planificación. Si Inventario tarda, el usuario espera antes de conocer si esa comprobación pudo realizarse. Una

consulta independiente de órdenes ya guardadas puede mantenerse disponible aunque esa integración falle; el fallo pertenece a una operación concreta.

Riesgo, alternativa y límite. Un valor de caché puede servir para una pantalla informativa si se muestra su antigüedad y el negocio admite desactualización. No sería correcto convertir esa caché en prueba de una reserva. Un fallback es una variante del contrato, no un truco para hacer desaparecer fallos del panel.

Comprobación técnica y aprendizaje. Mide duración del endpoint y duración de la llamada saliente con la misma correlación. Detén Inventario y verifica que la operación dependiente termina dentro de su límite y expresa indisponibilidad. Comprueba además una ruta sin esa dependencia para detectar un acoplamiento accidental de toda la API.

Bloque 6 — Riesgos de la comunicación síncrona

La comunicación síncrona introduce cuatro riesgos principales. El primero es el fallo en cascada: el fallo de un servicio se propaga a los servicios que dependen de él. El segundo es la latencia acumulada: la latencia del servicio origen incluye la latencia del servicio destino. El tercero es la saturación: si el servicio destino está saturado, el servicio origen también lo estará. El cuarto es la disponibilidad de la operación dependiente: esa operación no puede completar su camino normal si el destino no está disponible. Otras operaciones independientes del mismo servicio pueden seguir funcionando.

```
Riesgos de la comunicación síncrona:

Fallo en cascada.
Latencia acumulada.
Saturación.
Disponibilidad.
```

- **Línea 1:** `Riesgos de la comunicación síncrona:` → identifica el tema del ejemplo.
- **Línea 3:** `Fallo en cascada.` → primer riesgo.
- **Línea 4:** `Latencia acumulada.` → segundo riesgo.
- **Línea 5:** `Saturación.` → tercer riesgo.
- **Línea 6:** `Disponibilidad.` → cuarto riesgo.

Los riesgos tienen dos consecuencias prácticas. La primera es que la comunicación síncrona requiere políticas de resiliencia. La segunda es que la comunicación síncrona requiere monitorización. La combinación de ambas permite que los riesgos se mitiguen con timeouts, reintentos y circuit breakers.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Un fallo en cascada aparece cuando cada servicio conserva demasiadas solicitudes mientras espera a otro. Aunque await no bloquee un hilo durante E/S, las tareas pendientes consumen memoria, conexiones, tiempo y capacidad de concurrencia. La programación asíncrona no sustituye el control de carga.

Ejemplo concreto en AceriaApp. Imagina cien consultas simultáneas desde Producción hacia un Inventario lento. Si cada fallo provoca tres reintentos inmediatos, el destino puede recibir cientos de solicitudes adicionales justo cuando necesita recuperarse. El número de intentos debe analizarse junto con la concurrencia.

Riesgo, alternativa y límite. Los límites de concurrencia y colas de espera acotadas protegen recursos. Un circuit breaker rechaza temporalmente llamadas a una dependencia que falla; no arregla el servicio remoto ni prueba ausencia de stock. La métrica útil incluye rechazos, duración y recuperación, no solo excepciones.

Comprobación técnica y aprendizaje. Prueba un fallo controlado con carga moderada y observa cuántas llamadas salen realmente. Cuenta intentos por operación, mide p95 de latencia y comprueba que la cancelación detiene reintentos pendientes. Estos datos permiten distinguir una política protectora de una amplificación del incidente.

Bloque 7 — Mitigación de los riesgos

Los riesgos de la comunicación síncrona se mitigan con cuatro técnicas. La primera es el tiempo de espera: cada llamada tiene un tiempo máximo de espera. La segunda es el reintento: las llamadas fallidas se reintentan con retroceso exponencial. La tercera es el circuit breaker: el circuito se abre cuando el servicio destino falla de forma repetida. La cuarta es una respuesta alternativa explícita, si el negocio la admite. Un dato de caché debe señalar su antigüedad; inventar stock disponible como valor por defecto sería incorrecto.

```
using var limite = CancellationTokenSource.CreateLinkedTokenSource(ct);
limite.CancelAfter(TimeSpan.FromSeconds(2));

using var response = await httpClient.GetAsync("api/v1/materiales/CHAPA-3MM", limite.Token);
response.EnsureSuccessStatusCode();
```

- **Línea 1:** `using var limite = CancellationTokenSource.CreateLinkedTokenSource(ct);` → combina cancelación del llamador con un presupuesto local de espera.
- **Línea 2:** `limite.CancelAfter(TimeSpan.FromSeconds(2));` → solicita cancelación al agotarse dos segundos; no deshace trabajo remoto ya confirmado.
- **Línea 4:** `using var response = await httpClient.GetAsync("api/v1/materiales/CHAPA-3MM", limite.Token);` → propaga el presupuesto a la llamada y dispone la respuesta.
- **Línea 5:** `response.EnsureSuccessStatusCode();` → no transforma una respuesta de error en éxito; los reintentos se deciden por separado.

La mitigación de los riesgos tiene dos consecuencias prácticas. La primera es que la comunicación síncrona se vuelve más robusta. La segunda es que la comunicación síncrona se vuelve más compleja. La combinación de ambas permite que los riesgos se controlen a costa de añadir código.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Ordena las decisiones: presupuesto total, timeout por intento, errores transitorios, número máximo de reintentos, espera con jitter y límites de concurrencia. Un reintento es una nueva ejecución y exige saber si repetir la operación puede alterar el negocio.

Ejemplo concreto en AceriaApp. Un GET informativo puede reintentarse ante ciertos fallos transitorios, respetando cancelación y, cuando aplique, Retry-After. Un POST que planifica o reserva necesita una identidad de solicitud y comportamiento idempotente antes de reintentarse con seguridad. Un 400 por entrada inválida no mejora repitiéndolo.

Riesgo, alternativa y límite. El fragmento muestra presupuesto y cancelación, no una política completa de reintentos. Las APIs de resiliencia varían según el paquete y su versión; las prácticas son la referencia de las dependencias instaladas. No se presenta una llamada antigua de Polly como si fuera intercambiable con cualquier versión.

Comprobación técnica y aprendizaje. Comprueba que la operación termina dentro del presupuesto total y que no quedan solicitudes nuevas después de cancelar. Para una escritura, simula que el servidor confirma y la respuesta se pierde: el cliente debe reconocer el resultado como incierto o recuperar por identidad, en vez de repetir a ciegas.

Bloque 8 — Comunicación síncrona y consistencia

La comunicación síncrona no garantiza consistencia entre servicios. La operación se ejecuta en el servicio origen y en el servicio destino de forma independiente. Si una de las dos operaciones falla, la otra no se revierte automáticamente. La consistencia entre servicios requiere patrones adicionales como Saga o compensación. La elección de HTTP no determina por sí sola el nivel de consistencia. Puede usarse para lecturas y escrituras críticas, siempre que se diseñen las invariantes, la idempotencia y los límites transaccionales adecuados. Una Saga coordina transacciones locales y compensaciones, sin aportar una transacción ACID global.

```
Comunicación síncrona y consistencia:

Servicio A crea orden → Servicio B descuenta stock.
Si Servicio B falla → Servicio A debe compensar.
```

- **Línea 1:** `Comunicación síncrona y consistencia:` → identifica el tema del ejemplo.
- **Línea 3:** `Servicio A crea orden → Servicio B descuenta stock.` → operación distribuida.
- **Línea 4:** `Si Servicio B falla → Servicio A debe compensar.` → compensación manual.

El ejemplo de crear una orden y descontar stock es conceptual; no corresponde a una capacidad implementada del módulo. La consistencia tiene dos consecuencias prácticas. La primera es que la operación distribuida no es atómica. La segunda es que la compensación se implementa explícitamente. La combinación de ambas permite que el sistema mantenga la coherencia a costa de añadir complejidad.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La regla importante es qué datos deben cambiar juntos. Dos servicios tienen normalmente transacciones locales diferentes. Una respuesta HTTP de éxito de Inventario no hace atómico un commit posterior de Producción; un fallo entre ambos deja un estado intermedio que necesita tratamiento.

Ejemplo concreto en AceriaApp. Como ejemplo conceptual, reservar material y crear una orden podrían exigir una reserva identificada, su caducidad y una liberación compensatoria. Una compensación es otra operación de negocio, puede fallar y no borra necesariamente efectos externos ya observados. Esta alternativa no forma parte del recorrido implementado.

Riesgo, alternativa y límite. En el curso se consulta stock y luego se planifica localmente. Entre ambos instantes otra operación puede cambiar existencias: es una carrera de comprobación y uso. La promesa correcta es información observada, no garantía de material ni ausencia de sobreasignación.

Comprobación técnica y aprendizaje. Dibuja una línea temporal con dos peticiones que leen el mismo stock y después planifican. Si ambas cumplen la comprobación, has demostrado por qué una consulta previa no protege una invariante de inventario. Para imponerla habría que cambiar el contrato y el propietario de la reserva.



Interpretación del gráfico: Dos lectores pueden observar el mismo stock. Consultar antes de planificar no reserva unidades ni evita una carrera. La figura se distribuye en SVG vectorial y PNG.

Bloque 9 — Cuándo usar comunicación síncrona

La comunicación síncrona es una buena candidata cuando el siguiente paso necesita una respuesta actual y el presupuesto de latencia permite esperar. Es frecuente en lecturas, pero HTTP también admite escrituras. La sencillez del flujo puede justificarla incluso si se tolera consistencia eventual. Cuando no hace falta esperar al consumidor, la mensajería puede reducir dependencia temporal; no es una prohibición de usar HTTP. Ningún servicio remoto tiene disponibilidad perfecta: siempre hay que definir cómo se representa el fallo o el resultado desconocido.

```
Criterios para HTTP:

Resultado necesario ahora: candidato natural.
Lectura o escritura: ambas son posibles.
Latencia y disponibilidad: definir presupuesto y fallo.
Trabajo diferible: considerar mensajería.
Consistencia: diseñar invariantes, no inferirlas del protocolo.
```

- **Línea 1:** `Criterios para HTTP:` → identifica los criterios que guían la interacción.
- **Línea 3:** `Resultado necesario ahora: candidato natural.` → la dependencia del resultado favorece petición-respuesta.
- **Línea 4:** `Lectura o escritura: ambas son posibles.` → evita limitar HTTP a operaciones de lectura.
- **Línea 5:** `Latencia y disponibilidad: definir presupuesto y fallo.` → exige un contrato de latencia y comportamiento ante fallos.
- **Línea 6:** `Trabajo diferible: considerar mensajería.` → el procesamiento diferido puede reducir dependencia temporal.

- **Línea 7:** `Consistencia: diseñar invariantes, no inferirlas del protocolo.` → la consistencia depende de las fronteras de datos y coordinación.

La decisión tiene dos consecuencias prácticas. La primera es que la comunicación síncrona se aplica donde aporta valor. La segunda es que la comunicación asíncrona se aplica donde la síncrona no es adecuada. La combinación de ambas permite que el sistema elija el estilo adecuado a cada operación.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Formula la pregunta del actor antes de elegir transporte: «¿qué hay ahora?», «¿se aceptó mi solicitud?» y «¿terminó el trabajo?» son preguntas diferentes. Una respuesta HTTP puede contestar cualquiera si el contrato define su alcance, mientras un evento comunica un hecho a destinatarios desacoplados.

Ejemplo concreto en AceriaApp. Consultar una orden suele encajar en GET. Planificar una orden puede entrar por POST y devolver el resultado de una transacción local. La propagación posterior a Inventario puede viajar como evento sin cambiar que la entrada del usuario haya sido HTTP.

Riesgo, alternativa y límite. No conviertas los criterios en prohibiciones. Una lectura puede usar una proyección local eventual y una escritura puede usar HTTP con idempotencia. Una llamada sencilla de bajo riesgo puede seguir siendo apropiada aunque pudiera hacerse asíncrona, si el coste operativo adicional no se justifica.

Comprobación técnica y aprendizaje. Para cada interacción redacta una frase con el resultado esperado, el plazo y el fallo visible. Después compara si una respuesta inmediata o una aceptación diferida satisfacen esa frase. La decisión deja de depender de preferencias por una tecnología y se puede revisar con requisitos concretos.

Bloque 10 — Aplicación al proyecto del curso

El proyecto aplica comunicación síncrona entre la API de Producción heredada y el servicio separado Inventario.Api. La aplicación usa IClienteInventario; infraestructura implementa el cliente HTTP tipado. AddHttpClient centraliza URL, cabeceras y timeout. La comprobación devuelve información de existencias y distingue ausencia de recurso, respuesta inválida y fallo remoto. La validación combina pruebas del adaptador y ejecución HTTP entre procesos con SQL Server.

El punto prepara la publicación de eventos y el flujo híbrido. No cambia el propietario del stock ni garantiza material por consultar antes de planificar.

```
Aplicación al proyecto del curso:

Origen: API de Producción heredada de M5.
Destino: Inventario.Api, proceso separado.
Puerto: IClienteInventario.
Adaptador: cliente HTTP tipado con AddHttpClient.
Resultado: disponibilidad informativa, sin reserva.
Persistencia: SQL Server real en los contextos correspondientes.
```

- **Línea 1:** `Aplicación al proyecto del curso:` → presenta las responsabilidades del recorrido práctico.
- **Línea 3:** `Origen: API de Producción heredada de M5.` → mantiene el punto de entrada y el dominio de Producción.
- **Línea 4:** `Destino: Inventario.Api, proceso separado.` → introduce una frontera de proceso y de red.
- **Línea 5:** `Puerto: IClienteInventario.` → protege a la aplicación de los detalles del transporte.
- **Línea 6:** `Adaptador: cliente HTTP tipado con AddHttpClient.` → encapsula URL, estados HTTP y deserialización.

- **Línea 7:** `Resultado: disponibilidad informativa, sin reserva.` → no promete exclusión ni descuento de material.
- **Línea 8:** `Persistencia: SQL Server real en los contextos correspondientes.` → verifica persistencia con el proveedor real del curso.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El paso desde M5 conserva el dominio y la persistencia de Producción y añade una frontera real con Inventario. La arquitectura hexagonal sigue siendo útil: la aplicación depende del puerto y el adaptador implementa el transporte; no se mueve la lógica del agregado al controlador HTTP.

Ejemplo concreto en AceriaApp. La comprobación de disponibilidad debe ejercitarse con ambos procesos y SQL Server. Una respuesta de Inventario demuestra que se atravesó la red y se obtuvo un contrato; una consulta SQL posterior demuestra persistencia. Un test que solo enumera «HTTP» o «cliente tipado» no demuestra ese recorrido.

Riesgo, alternativa y límite. El punto no implementa una reserva de material. Un resultado disponible puede caducar inmediatamente. Esa limitación se conserva en los puntos siguientes: registrar una planificación en otro contexto no equivale a descontar stock, bloquear unidades o cumplir una promesa de entrega.

Comprobación técnica y aprendizaje. Verifica casos satisfactorios y de fallo con el servicio remoto activo, detenido y cancelado. Relaciona la evidencia con el requisito correspondiente y conserva los resultados para el cierre. Las prácticas ampliadas incluyen ejecución y diagnóstico; su duración depende del entorno y no se fija artificialmente en veintisiete minutos.

Resumen rápido de la teoría

- Una interacción síncrona espera un resultado; await no bloquea un hilo durante E/S.
- HTTP/1.1 y HTTP/2 usan TCP; HTTP/3 usa QUIC. HTTP admite lecturas y escrituras.
- El puerto expresa la necesidad local y el cliente tipado encapsula transporte, deserialización y errores.
- 404 indica ausencia por contrato; 500 no debe convertirse en material inexistente.
- Timeouts, cancelación, límites y reintentos seguros controlan riesgos, sin eliminar fallos.
- Consultar stock no reserva material ni garantiza consistencia entre servicios.

Punto 6.2 — Comunicación asíncrona basada en eventos (introducción)

Módulo 6: Comunicación entre microservicios **Objetivos de aprendizaje:**

- Comprender qué es la comunicación asíncrona basada en eventos.
- Identificar la diferencia entre eventos de dominio y eventos de integración.
- Comprender el papel del publicador y del consumidor de eventos.
- Comprender los beneficios y los costes de la comunicación asíncrona.
- Aplicar los conceptos al recorrido real de Producción e Inventario y verificar su comportamiento.

Desarrollo teórico

Bloque 1 — Definición de comunicación asíncrona

La comunicación asíncrona es un estilo de interacción en el que un servicio publica un mensaje y no espera una respuesta inmediata. El flujo de negocio no espera a que el consumidor termine. El publicador sí puede esperar una confirmación técnica del broker, y con `await` esa espera de E/S no bloquea un hilo. El mensaje se entrega a un intermediario que se encarga de distribuirlo a los consumidores. Los consumidores procesan el mensaje cuando pueden. El publicador y el consumidor no necesitan estar disponibles al mismo tiempo. La comunicación asíncrona se implementa habitualmente con un broker de mensajería.

```
Comunicación asíncrona:

Servicio A → publica evento → Broker → Servicio B consume.
Servicio A continúa sin esperar el efecto del consumidor.
```

- **Línea 1:** `Comunicación asíncrona:` → identifica el tema del ejemplo.
- **Línea 3:** `Servicio A → publica evento → Broker → Servicio B consume.` → flujo asíncrono.
- **Línea 4:** `Servicio A continúa sin esperar el efecto del consumidor.` → el productor no espera la finalización remota; la confirmación técnica tiene otro alcance.

La comunicación asíncrona se caracteriza por tres propiedades. La primera es que el publicador no espera el efecto de negocio del consumidor. La segunda es que el publicador y el consumidor no necesitan estar disponibles al mismo tiempo. La tercera es que el broker almacena y enruta mensajes según la topología y las políticas configuradas; la aplicación sigue siendo responsable de publicar, confirmar, reintentar y procesar correctamente. Las tres propiedades permiten que la interacción sea desacoplada temporalmente.

La aplicación de la comunicación asíncrona tiene dos consecuencias prácticas. La primera es que el publicador no depende de la disponibilidad del consumidor. La segunda es que el consumidor no necesita invocar una instancia concreta del publicador, aunque depende del significado, esquema y reglas de evolución de su contrato. La combinación de ambas permite que el sistema sea más resiliente y más evolutivo.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El desacoplamiento temporal desplaza la espera, no elimina dependencias. Producción no espera el efecto remoto, pero necesita confirmar su propio estado y conservar el evento pendiente. El publicador necesita después conectarse al broker, y el consumidor necesita su base de datos.

Ejemplo concreto en AceriaApp. Una orden planificada puede quedar confirmada mientras Inventario está detenido. Al volver el consumidor, el mensaje permite crear la proyección pendiente. La experiencia del usuario debe aceptar que la vista de Inventario vaya por detrás de la orden, y permitir identificar qué está pendiente.

Riesgo, alternativa y límite. Una publicación «fire and forget» en memoria puede perderse al terminar el proceso. El outbox hace duradera la intención dentro de la misma transacción local. No sustituye al worker que la envía ni a la capacidad de almacenamiento necesaria durante una interrupción prolongada.

Comprobación técnica y aprendizaje. Detén el consumidor, planifica una orden y comprueba que el estado local y el registro pendiente existen. Reactiva la entrega y observa la convergencia. Si el broker también está detenido, la prueba debe demostrar conservación en SQL, no fingir que un mensaje ya fue aceptado por RabbitMQ.

Bloque 2 — Eventos de dominio y eventos de integración

Los eventos se clasifican en dos tipos: eventos de dominio y eventos de integración. Los eventos de dominio representan hechos ocurridos dentro de un contexto delimitado. Los eventos de integración representan hechos que se comunican entre contextos delimitados. Los eventos de dominio se publican dentro del mismo servicio. Los eventos de integración se publican al broker para que otros servicios los consuman. La separación entre ambos permite que el modelo del dominio no dependa del broker.

```
Eventos de dominio    → dentro de un contexto.
Eventos de integración → entre contextos.
```

- **Línea 1:** `Eventos de dominio → dentro de un contexto.` → expresa el paso o la relación indicada en el ejemplo.
- **Línea 2:** `Eventos de integración → entre contextos.` → segundo tipo.

La distinción entre eventos de dominio y de integración tiene dos consecuencias prácticas. La primera es que el modelo del dominio no conoce el broker. La segunda es que los eventos de integración se diseñan como contratos públicos. La combinación de ambas permite que el dominio sea independiente de la infraestructura de mensajería.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Un evento de dominio expresa un hecho útil dentro del modelo; un evento de integración publica una parte estable de ese hecho para otros contextos. No es obligatorio serializar directamente el objeto interno. La traducción permite ocultar detalles que cambiarán con el diseño del agregado.

Ejemplo concreto en AceriaApp. Producción comunica OrdenPlanificadaV1 porque su transición se confirmó. Inventario necesita identificador de orden y líneas para construir una proyección. No necesita las navegaciones EF, los servicios de dominio ni toda la historia interna de la orden.

Riesgo, alternativa y límite. Publicar antes del commit permitiría que otro contexto actuara sobre un hecho que después se revierte. Publicar después sin outbox deja otra ventana: el proceso puede morir tras guardar y antes de enviar. Guardar estado y mensaje pendiente juntos cierra esa pérdida local de intención.

Comprobación técnica y aprendizaje. Revisa dependencias de proyecto y contenido del JSON. El dominio no debe referenciar RabbitMQ.Client. La prueba relevante fuerza un fallo de persistencia y confirma que no quedan una transición y un evento discordantes en la base local.

Bloque 3 — Estructura de un evento

Un evento tiene cuatro propiedades principales. La primera es el identificador único del evento. La segunda es el tipo de evento. La tercera es el momento en que ocurrió. La cuarta es la carga útil con los datos del evento. El identificador permite rastrear el evento. El tipo permite que los consumidores filtren los eventos que les interesan. El momento ayuda al diagnóstico y a medir retrasos; relojes distintos, resolución temporal y reentregas impiden usarlo como un orden global fiable. Si el negocio necesita orden por agregado, se añade una secuencia o versión y una política explícita. La carga útil contiene la información necesaria para que el consumidor reaccione.

```
public sealed record LineaPlanificadaV1(string Material, int Cantidad);

public sealed record OrdenPlanificadaV1(
    Guid EventoId,
    Guid OrdenId,
    DateTimeOffset OcurridoEnUtc,
    IReadOnlyList<LineaPlanificadaV1> Lineas,
    int Version = 1);
```

- **Línea 1:** `public sealed record LineaPlanificadaV1(string Material, int Cantidad);` → representa una línea del contrato compartido de planificación.
- **Línea 3:** `public sealed record OrdenPlanificadaV1(` → nombra el hecho y la versión de su contrato de integración.
- **Línea 4:** `Guid EventoId,` → mantiene identidad estable durante los reintentos del mismo evento.
- **Línea 5:** `Guid OrdenId,` → correlaciona la publicación con la orden de Producción.
- **Línea 6:** `DateTimeOffset OcurridoEnUtc,` → expresa el instante con zona UTC; no constituye un orden global.
- **Línea 7:** `IReadOnlyList<LineaPlanificadaV1> Lineas,` → incluye las líneas necesarias para construir la proyección sin consultar tablas ajenas.
- **Línea 8:** `int Version = 1);` → declara la versión admitida del contrato; el consumidor valida este valor.

El evento se caracteriza por tres propiedades. La primera es que es inmutable. La segunda es que contiene la información necesaria para que el consumidor reaccione. La tercera es que su nombre expresa un hecho ya ocurrido, como `OrdenPlanificada`, y no una instrucción como `PlanificarOrden`. Las tres propiedades permiten que el evento sea un contrato estable y explícito.

Estos contratos pertenecen a `AceriaApp.Comunicacion.Contratos`. El consumidor valida `Version`, la identidad y el contenido; una versión incompatible exige una estrategia de convivencia. La clave primaria por EventoId, el índice único por OrdenId y la huella del contenido permiten detectar duplicados y rechazar una identidad reutilizada con otro payload.

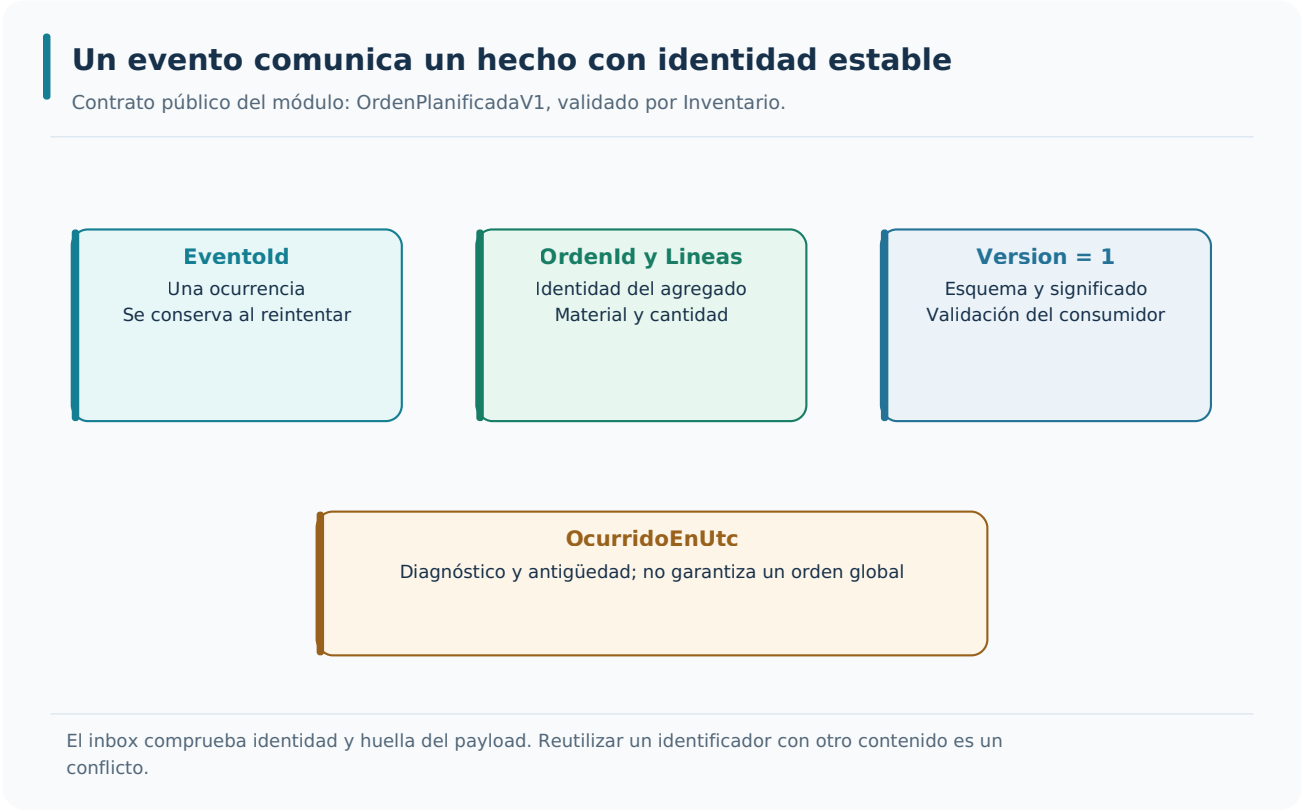
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La identidad del evento representa una ocurrencia, no un intento de transporte. Reenviar la misma fila de outbox debe conservar EventoId. Si se genera otro Guid al reintentar, el inbox verá un hecho diferente y no podrá deduplicarlo con esa identidad.

Ejemplo concreto en AceriaApp. OrdenPlanificadaV1 expresa el hecho y su versión. OrdenId permite correlacionarlo con el agregado; las líneas aportan el contenido que Inventario necesita. El instante UTC ayuda a medir antigüedad y diagnóstico, mientras una secuencia por orden sería necesaria si futuros eventos exigieran aplicar cambios en orden.

Riesgo, alternativa y límite. Un record reduce mutaciones accidentales, pero IReadOnlyList no vuelve inmutables los elementos de una lista externa por arte de magia. Construye una instantánea antes de serializar. La compatibilidad depende también de nombres JSON, campos obligatorios y significado, no solo del sufijo V1.

Comprobación técnica y aprendizaje. Serializa, deserializa y compara la identidad y los datos de una orden con varias líneas. Repite la publicación y verifica que EventoId se conserva. Introduce un campo desconocido y documenta qué evolución admite el consumidor, evitando suponer que todas las modificaciones son compatibles.



Interpretación del gráfico: EventoId identifica el hecho; OrdenId correlaciona el agregado; la versión y el contenido forman un contrato estable. La figura se distribuye en SVG vectorial y PNG.

Bloque 4 — Publicador de eventos

El publicador envía eventos al broker desde infraestructura y puede implementar un puerto de salida como IPublicadorEventos definido en aplicación. Convierte el contrato en JSON y metadatos; no conoce las instancias consumidoras. La decisión de publicar un hecho confirmado nace del caso de uso, y el outbox conserva esa intención junto con el estado de Producción.

El fragmento ilustra la parte de transporte con RabbitMQ.Client 7.1.2. Presupone una conexión administrada y topología declarada: exchange, cola durable y binding. El trabajador conserva el EventoId, espera confirms y solo después marca la fila como enviada. La conexión y los canales necesitan una política de ciclo de vida y concurrencia explícita. La clave de enrutamiento y el tipo coinciden con la práctica. El fragmento añade `MessageId` como metadato opcional; el receptor implementado obtiene `EventoId` del contrato JSON para deduplicar.

```
var opciones = new CreateChannelOptions(
    publisherConfirmationsEnabled: true,
    publisherConfirmationTrackingEnabled: true);
await using var canal = await conexion.CreateChannelAsync(opciones, ct);
var cuerpo = JsonSerializer.SerializeToUtf8Bytes(evento);
var propiedades = new BasicProperties
{
    Persistent = true,
    ContentType = "application/json",
    MessageId = evento.EventoId.ToString(),
    Type = "orden.planificada.v1"
};
await canal.BasicPublishAsync(
    exchange: "aceria.eventos",
    routingKey: "orden.planificada.v1",
    mandatory: true,
    basicProperties: propiedades,
    body: cuerpo,
    cancellationToken: ct);
```

- **Línea 1:** `var opciones = new CreateChannelOptions(` → configura el canal del cliente RabbitMQ.Client 7.1.2.
- **Línea 2:** `publisherConfirmationsEnabled: true,` → activa las confirmaciones del broker.
- **Línea 3:** `publisherConfirmationTrackingEnabled: true);` → activa su seguimiento para esperar y detectar confirmación negativa o devolución.
- **Línea 4:** `await using var canal = await conexion.CreateChannelAsync(opciones, ct);` → crea y dispone un canal asíncrono; la conexión pertenece al componente de infraestructura.
- **Línea 5:** `var cuerpo = JsonSerializer.SerializeToUtf8Bytes(evento);` → serializa el contrato con la identidad persistida en el outbox.
- **Línea 6:** `var propiedades = new BasicProperties` → construye metadatos del mensaje.
- **Línea 7:** `{` → abre el bloque de la declaración anterior.
- **Línea 8:** `Persistent = true,` → solicita persistencia; también se necesitan cola durable y confirmación.
- **Línea 9:** `ContentType = "application/json",` → describe el formato de la carga útil.
- **Línea 10:** `MessageId = evento.EventoId.ToString(),` → conserva el identificador del evento entre reenvíos.
- **Línea 11:** `Type = "orden.planificada.v1"` → expresa una versión pública estable, sin depender de un renombrado accidental de clase.
- **Línea 12:** `};` → cierra el bloque y, cuando corresponde, la instrucción.
- **Línea 13:** `await canal.BasicPublishAsync(` → publica y espera confirmación técnica con las opciones anteriores.
- **Línea 14:** `exchange: "aceria.eventos",` → usa el exchange declarado previamente.
- **Línea 15:** `routingKey: "orden.planificada.v1",` → selecciona la suscripción mediante una clave explícita.
- **Línea 16:** `mandatory: true,` → solicita detectar mensajes que no se pueden enrutar a ninguna cola.
- **Línea 17:** `basicProperties: propiedades,` → asocia persistencia e identidad a la publicación.
- **Línea 18:** `body: cuerpo,` → envía el contenido serializado.

- **Línea 19:** `cancellationToken: ct);` → propaga la cancelación; un resultado incierto conserva la fila pendiente del outbox.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El publicador de infraestructura transforma el contrato en una operación del broker. El worker del outbox decide qué registros siguen pendientes y cuándo puede marcarlos como enviados. Esperar una confirmación técnica no significa que Inventario haya ejecutado su transacción.

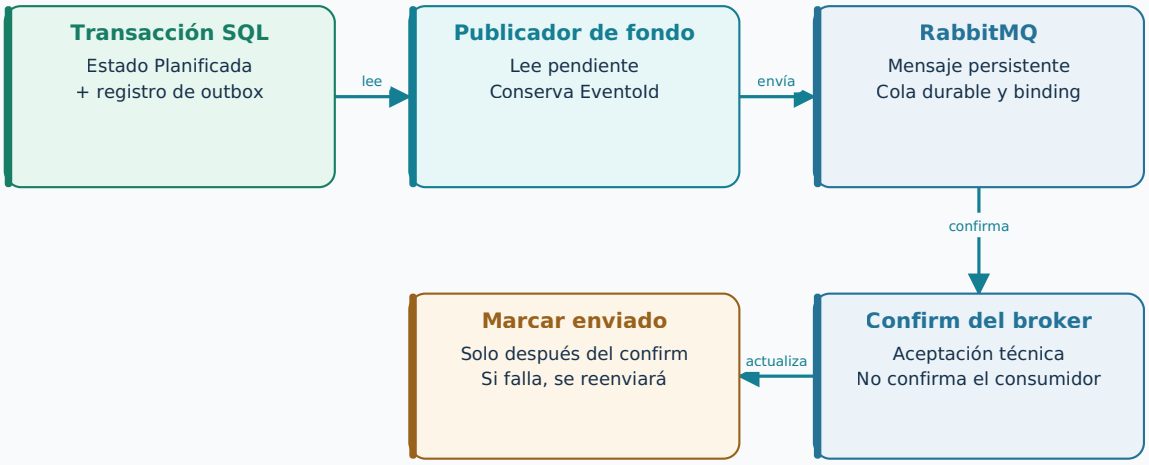
Ejemplo concreto en AceriaApp. La práctica usa RabbitMQ.Client 7.1.2 con IChannel y métodos Async. El ejemplo muestra confirms, persistencia, identidad y publicación mandatory; presupone exchange, cola durable y binding ya declarados. En producción interesa reutilizar conexiones y administrar canales sin uso concurrente inseguro.

Riesgo, alternativa y límite. Existe una ventana inevitable entre recibir confirmación y marcar el outbox. Si el proceso cae ahí, reenviará el evento al reiniciar. Ese duplicado es preferible a perder la intención y explica por qué el consumidor necesita inbox. La recuperación automática de conexión tampoco reenvía por sí sola todos los mensajes inciertos.

Comprobación técnica y aprendizaje. Prueba el recorrido con el broker caído y después disponible. Comprueba que no se marca enviado al fallar, que el mismo EventoId llega tras reintentar y que una ruta sin cola no pasa por éxito silencioso. No basta con que BasicPublishAsync retorne si el canal no tiene configuradas las confirmaciones que el diseño presupone.

Outbox: conservar el hecho antes de publicarlo

La intención de integración comparte el commit local con la planificación.



Una caída tras el confirm y antes de la marca SQL genera reenvío. La idempotencia del receptor cubre ese duplicado.

Interpretación del gráfico: Estado y outbox se confirman juntos. El confirm del broker precede a marcar enviado, y esa ventana admite duplicados. La figura se distribuye en SVG vectorial y PNG.

Bloque 5 — Consumidor de eventos

El consumidor recibe mensajes del broker, valida el contrato y coordina un manejador de aplicación. No necesita invocar al publicador para interpretar un hecho autosuficiente, aunque sigue dependiendo de su contrato. Puede haber varios manejadores o suscripciones; cada efecto necesita su política de consistencia y recuperación.

En el curso, OrdenPlanificadaV1 crea una proyección de planificación en Inventario. Inbox y proyección se guardan en una transacción SQL local y el ack se envía después. El algoritmo describe la frontera de idempotencia. Como alternativa conceptual, un evento de creación podría iniciar una reserva o facturación, pero esos efectos requieren reglas adicionales y no están implementados aquí.

```
Recepción y efecto local:

1. Deserializar y validar OrdenPlanificadaV1.
2. Iniciar una transacción SQL en Inventario.
3. Registrar EventoId en inbox con unicidad.
4. Guardar la proyección de planificación.
5. Confirmar la transacción local.
6. Enviar ack manual en el canal receptor.
Duplicado confirmado: reconocer sin repetir el efecto.
```

- **Línea 1:** `Recepción y efecto local:` → describe el algoritmo; el receptor real usa la API asíncrona del cliente.
- **Línea 3:** `1. Deserializar y validar OrdenPlanificadaV1.` → valida esquema y campos antes de tocar el estado.
- **Línea 4:** `2. Iniciar una transacción SQL en Inventario.` → define una unidad atómica para deduplicación y proyección.
- **Línea 5:** `3. Registrar EventoId en inbox con unicidad.` → la restricción única protege también frente a consumidores concurrentes.
- **Línea 6:** `4. Guardar la proyección de planificación.` → registra información de planificación; no modifica existencias.
- **Línea 7:** `5. Confirmar la transacción local.` → hace duraderos inbox y proyección juntos.
- **Línea 8:** `6. Enviar ack manual en el canal receptor.` → solo después del commit permite al broker retirar la entrega.
- **Línea 9:** `Duplicado confirmado: reconocer sin repetir el efecto.` → una reentrega puede resolverse sin duplicar la proyección.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El consumidor tiene dos responsabilidades distintas: interpretar el mensaje y confirmar un efecto local. La recepción no es el éxito de negocio. El ack manual debe enviarse después del commit; enviarlo antes permite perder el trabajo si el proceso muere antes de guardarlo.

Ejemplo concreto en AceriaApp. Inventario registra una proyección de la planificación, junto con EventoId en su inbox SQL. Al recibir el mismo hecho otra vez, reconoce que ya aplicó el efecto y no duplica la vista. No reserva ni descuenta stock: una proyección informativa conserva la semántica del módulo.

Riesgo, alternativa y límite. Un simple «si no existe, insertar» sin unicidad tiene una carrera entre consumidores. La restricción de base de datos y el manejo del conflicto son necesarios. Inbox y proyección deben compartir transacción; si se guardan separados, un fallo entre ambos puede dejar un evento marcado sin efecto.

Comprobación técnica y aprendizaje. Reenvía el mismo mensaje y comprueba una sola proyección y una sola identidad procesada. Simula caída tras commit y antes del ack: la reentrega debe ser segura. El recorrido usa BasicGetAsync con sondeo didáctico y ack manual, no suscripción push. Si se amplía a AsyncEventingBasicConsumer, el cuerpo debe deserializarse o copiarse dentro del callback antes de conservarlo fuera de su memoria válida.

Bloque 6 — Broker de mensajería

El broker de mensajería es el intermediario que recibe los eventos del publicador y los entrega a los consumidores. El broker ofrece mecanismos de entrega y durabilidad que deben configurarse y combinarse con confirmaciones y recuperación de la aplicación; no garantiza por sí solo la entrega ni el efecto de negocio. El broker desacopla al publicador del consumidor. Para que varias suscripciones reciban una copia, se emplean colas diferentes enlazadas al exchange. Varias instancias que comparten una cola compiten por el trabajo: no reciben todas cada mensaje. Los brokers más usados en el ecosistema .NET son RabbitMQ, Azure Service Bus y Kafka. La elección depende de los requisitos de entrega, orden y persistencia.

```
Topología de publicación y suscripción:

Producción publica en un exchange.
Cola durable de Inventario: una copia por evento enrutado.
Instancias de Inventario en esa cola: consumidores competidores.
Otra suscripción necesita otra cola y su enlace.
Confirmación del broker y ack del consumidor son distintos.
```

- **Línea 1:** `Topología de publicación y suscripción:` → separa el mecanismo de distribución de sus garantías.
- **Línea 3:** `Producción publica en un exchange.` → el exchange enruta, no sustituye al almacenamiento de las colas.
- **Línea 4:** `Cola durable de Inventario: una copia por evento enrutado.` → una cola durable mantiene su definición al reiniciar el broker.
- **Línea 5:** `Instancias de Inventario en esa cola: consumidores competidores.` → las instancias reparten trabajo; no forman una difusión por instancia.
- **Línea 6:** `Otra suscripción necesita otra cola y su enlace.` → cada suscripción independiente tiene su propia copia.
- **Línea 7:** `Confirmación del broker y ack del consumidor son distintos.` → confirmar publicación no significa haber confirmado el efecto SQL remoto.

El broker se caracteriza por tres propiedades. La primera es que proporciona almacenamiento, enrutamiento y confirmaciones bajo condiciones de operación definidas. La segunda es que desacopla al publicador del consumidor. La tercera es que permite varios consumidores. Las tres propiedades permiten que el sistema sea escalable y resiliente.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. En RabbitMQ, exchange, binding y cola tienen responsabilidades diferentes. El exchange aplica reglas de enrutamiento; la cola conserva mensajes y los ofrece a consumidores. La durabilidad de la cola conserva su definición, mientras la persistencia del mensaje expresa cómo debe almacenarse su contenido.

Ejemplo concreto en AceriaApp. Inventario tiene una suscripción al hecho de planificación. Si ejecutas dos instancias sobre la misma cola, comparten el trabajo. Si en una ampliación conceptual Facturación necesitara una copia independiente, tendría su propia cola y su contrato de consumo; no basta añadir otra instancia a la cola de Inventario.

Riesgo, alternativa y límite. RabbitMQ, Azure Service Bus y Kafka ofrecen modelos y garantías diferentes. Comparar sus nombres no elige una topología. Deben evaluarse retención, repetición, orden, disponibilidad, permisos y operación; una cola de trabajo y un log particionado no son intercambiables sin rediseñar el consumidor.

Comprobación técnica y aprendizaje. Inspecciona la topología y verifica qué ocurre cuando falta el binding. Después arranca dos consumidores de la misma cola y observa reparto, no difusión. Estas pruebas distinguen la distribución de mensajes de la deduplicación de efectos en SQL.

Bloque 7 — Garantías de entrega

Las semánticas de entrega se describen como máximo una vez, al menos una vez y exactamente una vez, siempre indicando el ámbito y los supuestos. Como máximo una vez admite pérdida y evita reintentos. Al menos una vez utiliza confirmaciones y reintentos, por lo que puede producir duplicados. Exactamente una vez no es una promesa general de RabbitMQ para los efectos de una aplicación. Algunas plataformas ofrecen garantías acotadas a su propio almacenamiento o protocolo; un efecto SQL o externo requiere coordinación adicional. El recorrido del curso se diseña para reentrega e idempotencia, sujeto a conservación de datos y recuperación de los componentes.

```
Semánticas y límites:

Como máximo una vez: puede perderse el mensaje.
Al menos una vez: diseñar para duplicados y recuperación.
Exactamente una vez: especificar ámbito; no asumir efecto global.
Curso: outbox + confirms + ack manual + inbox transaccional.
```

- **Línea 1:** `Semánticas y límites:` → obliga a nombrar el alcance de una garantía.
- **Línea 3:** `Como máximo una vez: puede perderse el mensaje.` → evitar reintentos puede evitar duplicados, pero admite pérdida.
- **Línea 4:** `Al menos una vez: diseñar para duplicados y recuperación.` → la reentrega requiere operaciones idempotentes.
- **Línea 5:** `Exactamente una vez: especificar ámbito; no asumir efecto global.` → no se atribuye una garantía global al broker.
- **Línea 6:** `Curso: outbox + confirms + ack manual + inbox transaccional.` → combina mecanismos de transporte con atomicidad y deduplicación locales.

Las garantías de entrega tienen dos consecuencias prácticas. La primera es que la entrega como mínimo una vez requiere idempotencia en el consumidor. La segunda es que un efecto local único requiere deduplicación persistente y una transacción que incluya el efecto. Un registro inbox separado de la actualización de negocio deja una ventana de pérdida o duplicación. La combinación de ambas permite que el sistema elija la garantía adecuada a cada operación.

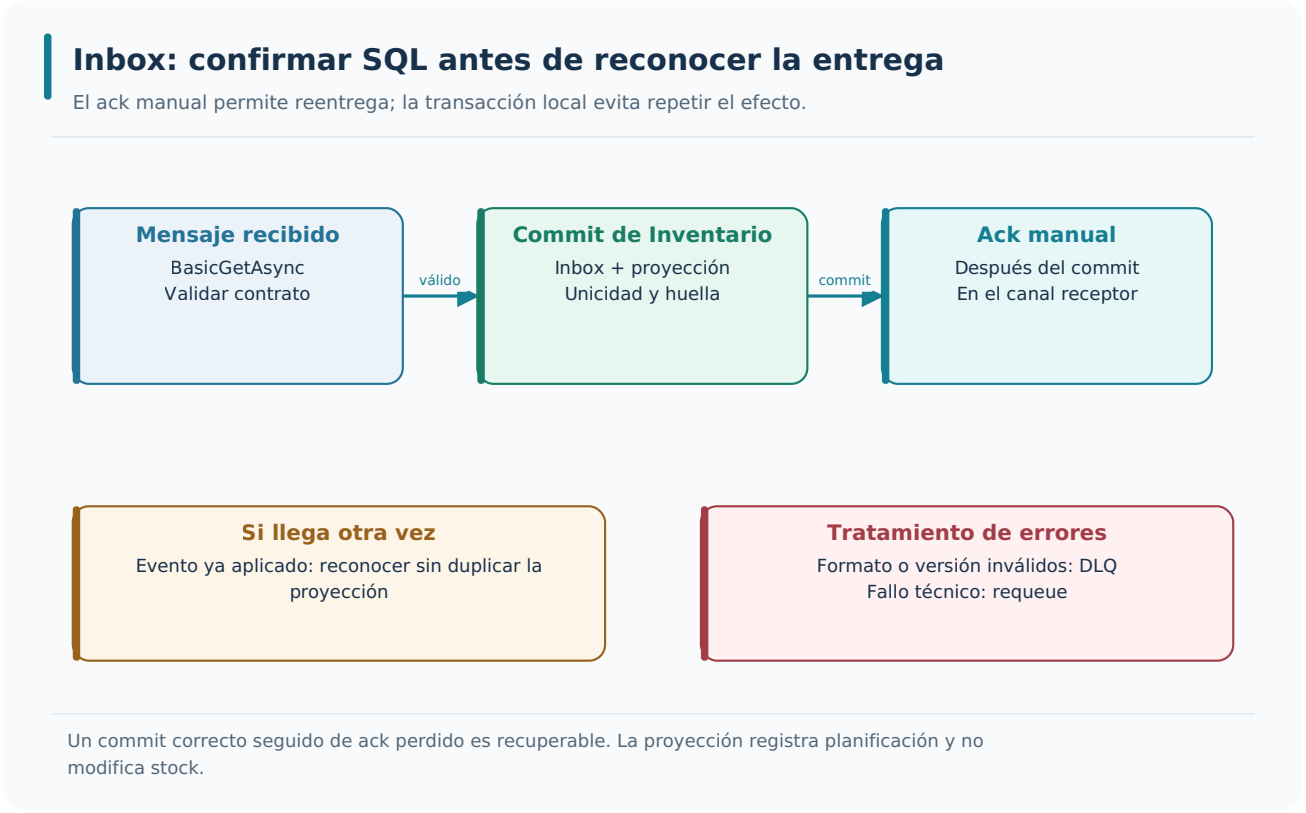
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Las confirmaciones del publicador cubren el tramo hacia el broker; los acknowledgements del consumidor cubren el tramo de entrega hacia la aplicación. Ninguna de ellas convierte automáticamente dos bases de datos y un broker en una transacción única.

Ejemplo concreto en AceriaApp. El caso más instructivo es «commit correcto, ack perdido». RabbitMQ puede entregar de nuevo y el inbox evita repetir la proyección. En el lado del publicador, «confirm recibido, marca de outbox no guardada» también genera un reenvío. Ambos casos deben compartir la misma identidad del evento.

Riesgo, alternativa y límite. La expresión «exactamente una vez» necesita un ámbito concreto. La implementación busca un único efecto local para un EventoId mediante unicidad y transacción, bajo sus supuestos de almacenamiento. No promete una entrega única ni que cualquier llamada externa realizada dentro del handler sea automáticamente idempotente.

Comprobación técnica y aprendizaje. Construye una tabla de fallos antes y después de cada commit y confirmación. Para cada fila explica si se reintenta, se deduplica o se requiere intervención. Una prueba de duplicado demuestra una propiedad acotada; no demuestra por sí sola recuperación frente a pérdida de disco o restauraciones inconsistentes.



Interpretación del gráfico: Inbox y proyección comparten commit. Una caída después del commit y antes del ack provoca reentrega segura mediante deduplicación. La figura se distribuye en SVG vectorial y PNG.

Bloque 8 — Beneficios de la comunicación asíncrona

La comunicación asíncrona aporta cuatro beneficios principales. El primero es el desacoplamiento temporal: el publicador y el consumidor no necesitan estar disponibles al mismo tiempo. El segundo es el desacoplamiento de conocimiento: el publicador no conoce a los consumidores. El tercero es la escalabilidad: los consumidores se escalan de forma independiente. El cuarto es la resiliencia frente a una interrupción temporal del consumidor: el publicador puede continuar mientras el outbox y el broker tengan capacidad. Una cola llena, el disco agotado o la caída de la base local sí afectan al sistema. Los cuatro beneficios se materializan cuando el sistema tiene operaciones que no requieren respuesta inmediata.

```
Beneficios de la comunicación asíncrona:

Desacoplamiento temporal.
Desacoplamiento de conocimiento.
Escalabilidad.
Resiliencia.
```

- **Línea 1:** `Beneficios de la comunicación asíncrona:` → identifica el tema del ejemplo.
- **Línea 3:** `Desacoplamiento temporal.` → primer beneficio.
- **Línea 4:** `Desacoplamiento de conocimiento.` → segundo beneficio.
- **Línea 5:** `Escalabilidad.` → tercer beneficio.
- **Línea 6:** `Resiliencia.` → cuarto beneficio.

Los beneficios tienen dos consecuencias prácticas. La primera es que el sistema se adapta mejor a las cargas desiguales. La segunda es que el sistema tolera mejor los fallos. La combinación de ambas permite que el sistema sea más robusto.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Una cola permite desacoplar ritmo de llegada y ritmo de trabajo durante un intervalo. Esa elasticidad es útil cuando Inventario puede ponerse al día después de un pico. El beneficio se mide por retraso tolerable y recuperación, no solo por haber introducido un broker.

Ejemplo concreto en AceriaApp. Si llegan veinte planificaciones por segundo durante un minuto y el consumidor procesa treinta, puede absorber la carga. Si llegan cuarenta de forma sostenida, el pendiente crecerá. Escalar consumidores ayuda solo mientras SQL, el trabajo y las claves de concurrencia admitan esa paralelización.

Riesgo, alternativa y límite. El desacoplamiento de conocimiento elimina la llamada directa a un destinatario concreto, pero conserva dependencia de contrato. Un evento que cambia de significado puede romper consumidores aunque el productor nunca conozca sus direcciones. La autonomía necesita compatibilidad y propiedad explícita.

Comprobación técnica y aprendizaje. Mide edad del evento más antiguo, tasa de entrada y salida, tamaño de outbox y tiempo de recuperación. Detener y reanudar el consumidor debería mostrar acumulación y vaciado. Si el pendiente nunca disminuye, la solución necesita capacidad o reducción de carga, no solo más reintentos.

Bloque 9 — Costes de la comunicación asíncrona

La comunicación asíncrona también tiene costes. El primer coste es la complejidad: el sistema requiere un broker, consumidores y mecanismos de reintento. El segundo coste es la consistencia eventual: el consumidor procesa el evento después de que el publicador lo haya emitido, por lo que el estado del sistema puede ser temporalmente inconsistente. El tercer coste es la idempotencia: la entrega como mínimo una vez requiere que el consumidor procese el mismo evento varias veces sin efectos duplicados. El cuarto coste es la observabilidad: rastrear un evento a través del broker y de varios consumidores requiere herramientas específicas.

```
Costes de la comunicación asíncrona:

Complejidad.
Consistencia eventual.
Idempotencia.
Observabilidad.
```

- **Línea 1:** `Costes de la comunicación asíncrona:` → identifica el tema del ejemplo.
- **Línea 3:** `Complejidad.` → primer coste.
- **Línea 4:** `Consistencia eventual.` → segundo coste.
- **Línea 5:** `Idempotencia.` → tercer coste.
- **Línea 6:** `Observabilidad.` → cuarto coste.

Los costes tienen dos consecuencias prácticas. La primera es que la comunicación asíncrona no se aplica en todos los contextos. La segunda es que su aplicación se concentra donde aporta valor. La combinación de ambas permite que el esfuerzo se invierta donde produce beneficio.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La consistencia eventual significa que una vista derivada puede retrasarse respecto de la fuente y debe converger cuando el sistema se recupera. No equivale a

aceptar cualquier divergencia indefinida. El contrato debe definir un plazo razonable y cómo detectar una planificación que no progresa.

Ejemplo concreto en AceriaApp. En el curso, la orden puede estar planificada y la proyección de Inventario aún ausente. La interfaz debe distinguir «pendiente de propagación» de «orden inexistente». El estado visible depende del recurso consultado y del instante, por lo que conviene conservar identificadores de correlación.

Riesgo, alternativa y límite. Un mensaje inválido reencolado continuamente puede ocupar al consumidor e impedir progreso. Una operación real necesita límite de intentos, clasificación entre fallo transitorio y permanente y un camino de diagnóstico o reproceso. Las capacidades no implementadas deben describirse como trabajo adicional, no como garantías de una biblioteca.

Comprobación técnica y aprendizaje. Revisa qué evidencia une solicitud HTTP, OrdenId, EventoId, fila de outbox y registro inbox. Evita registrar cuerpos completos con información innecesaria. Un diagnóstico útil permite responder qué se confirmó, qué sigue pendiente y dónde debe actuar el operador.

Bloque 10 — Aplicación al proyecto del curso

El proyecto aplica comunicación asíncrona a la propagación de la planificación. Producción guarda su cambio de estado y un evento pendiente en el outbox dentro de la misma transacción SQL. Un trabajador publica OrdenPlanificadaV1 con RabbitMQ. Inventario.Api consume, deduplica mediante inbox y registra la proyección en su propia transacción.

La comunicación se prueba con broker real, mensajes persistentes, cola durable, confirms y ack manual. Los ejemplos conceptuales de OrdenCreada, OrdenCancelada, reserva, liberación y Facturacion.Api permiten discutir alternativas, pero no forman parte de este recorrido. El punto siguiente decide dónde conviene combinar consulta inmediata y propagación eventual.

```
Aplicación al proyecto del curso:

Producción confirma estado y outbox en la misma transacción SQL.
Un publicador en segundo plano envía OrdenPlanificadaV1.
RabbitMQ enruta mensajes persistentes a una cola durable.
Inventario.Api confirma inbox y proyección en su transacción SQL.
La lectura de la proyección es eventual.
No se reserva stock ni se implementa facturación.
```

- **Línea 1:** `Aplicación al proyecto del curso:` → delimita el recorrido implementado.
- **Línea 3:** `Producción confirma estado y outbox en la misma transacción SQL.` → evita perder la intención de integración tras guardar el estado.
- **Línea 4:** `Un publicador en segundo plano envía OrdenPlanificadaV1.` → reintenta la publicación manteniendo EventoId.
- **Línea 5:** `RabbitMQ enruta mensajes persistentes a una cola durable.` → requiere además confirms, rutas correctas y almacenamiento disponible.
- **Línea 6:** `Inventario.Api confirma inbox y proyección en su transacción SQL.` → vincula la deduplicación con el efecto local.
- **Línea 7:** `La lectura de la proyección es eventual.` → puede existir retraso entre ambas bases.
- **Línea 8:** `No se reserva stock ni se implementa facturación.` → mantiene las alternativas de negocio fuera del alcance acreditado.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La integración añade dos commits locales unidos por mensajería. Producción confirma su estado y la intención de publicación; Inventario confirma la deduplicación y la vista. El periodo entre ambos es parte del diseño y debe ser visible en pruebas y documentación.

Ejemplo concreto en AceriaApp. El recorrido publica OrdenPlanificadaV1 y registra su proyección. Un ejemplo de OrdenCreada u OrdenCancelada para reservar, liberar o facturar puede servir para discutir un diseño futuro, pero exigiría nuevas reglas, contratos, permisos, compensaciones y pruebas. Aquí no acredita capacidades existentes.

Riesgo, alternativa y límite. No deduzcas recuperación completa porque una cola sea durable. El outbox necesita un publicador activo; el consumidor necesita SQL disponible; las confirmaciones pueden perderse y generar duplicados. Incluso en el recorrido satisfactorio deben comprobarse la identidad del mensaje y el orden de los commits.

Comprobación técnica y aprendizaje. Obtén evidencia de estado local, fila pendiente, publicación confirmada, inbox y proyección. Repite el evento y detén temporalmente un componente. El cierre debe indicar qué escenarios se ejecutaron realmente y separar pruebas de comportamiento de validadores que solo describen el catálogo arquitectónico.

Resumen rápido de la teoría

- El productor no espera el efecto del consumidor, pero sí conserva y confirma su intención local.
- Eventos de dominio e integración tienen ámbitos distintos; el contrato público se versiona.
- EventoId identifica el hecho y se conserva al reintentar; un timestamp no ordena globalmente.
- Outbox vincula estado e intención; confirms verifican el tramo hasta el broker.
- Inbox y efecto local comparten transacción; el ack se envía después del commit.
- Las reentregas son posibles. Durabilidad y deduplicación no prometen un efecto global exactamente una vez.
- La práctica proyecta una planificación en Inventario sin reserva ni facturación.

Punto 6.3 — Cuándo utilizar cada enfoque

Módulo 6: Comunicación entre microservicios **Objetivos de aprendizaje:**

- Comprender los criterios para elegir entre comunicación síncrona y asíncrona.
- Identificar los factores técnicos y organizativos que influyen en la decisión.
- Comprender los patrones híbridos que combinan ambos enfoques.
- Comprender el impacto de la decisión en la consistencia y la resiliencia.
- Aplicar los conceptos al recorrido real de Producción e Inventario y verificar su comportamiento.

Desarrollo teórico

Bloque 1 — La decisión como parte del diseño arquitectónico

La elección entre comunicación síncrona y asíncrona es una decisión arquitectónica que se toma en el momento de diseñar la interacción entre dos servicios. La decisión no es binaria: un mismo sistema puede usar comunicación síncrona para unas operaciones y asíncrona para otras. La decisión se basa en criterios técnicos y organizativos que se evalúan para cada interacción. La decisión se documenta

en un ADR para que sea trazable. La decisión se revisa cuando cambian los requisitos del negocio o de la infraestructura.

```
Decisión de comunicación:

Interacción A → comunicación síncrona.
Interacción B → comunicación asíncrona.
Interacción C → comunicación híbrida.
```

- **Línea 1:** `Decisión de comunicación:` → identifica el tema del ejemplo.
- **Línea 3:** `Interacción A → comunicación síncrona.` → primera interacción.
- **Línea 4:** `Interacción B → comunicación asíncrona.` → segunda interacción.
- **Línea 5:** `Interacción C → comunicación híbrida.` → tercera interacción.

La decisión se caracteriza por tres propiedades. La primera es que se toma por interacción y no por sistema. La segunda es que se basa en criterios explícitos. La tercera es que se documenta para que sea trazable. Las tres propiedades permiten que la elección sea razonada y que se revise cuando cambien las circunstancias.

La aplicación de la decisión tiene dos consecuencias prácticas. La primera es que el sistema se adapta a las necesidades de cada interacción. La segunda es que la arquitectura es más flexible. La combinación de ambas permite que el sistema sea coherente y mantenible.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La unidad de decisión es una interacción con una necesidad concreta. Elegir «todos los servicios usan eventos» oculta diferencias entre consultar un dato, aceptar una intención y notificar un hecho. Una misma orden puede recorrer las tres formas a lo largo de su vida.

Ejemplo concreto en AceriaApp. En AceriaApp, consultar existencias es una interacción inmediata; propagar la planificación es una interacción diferida; el endpoint que combina consulta y commit local forma un flujo híbrido. La coherencia consiste en que cada contrato declara qué sabe y qué promete.

Riesgo, alternativa y límite. Un ADR debe incluir contexto, alternativas, decisión, consecuencias y condiciones de revisión. No necesita una lista de modas arquitectónicas. Si el requisito cambiara a «garantizar material disponible», habría que revisar la frontera de reserva, aunque HTTP y RabbitMQ siguieran disponibles.

Comprobación técnica y aprendizaje. Lee el ADR desde la perspectiva de otro equipo: debe poder identificar el dato autoritativo, la latencia admitida y qué ocurre al fallar. Si solo afirma que un enfoque es «más escalable», añade una situación concreta y la evidencia necesaria para validar esa ventaja.

Bloque 2 — Criterios técnicos: necesidad de respuesta inmediata

El primer criterio técnico es qué respuesta necesita el actor y cuándo. Si el siguiente paso necesita el resultado del destino en la misma interacción, una llamada HTTP es una candidata natural. Si basta confirmar aceptación duradera, el trabajo posterior puede ser asíncrono. Un usuario puede aceptar una pantalla «pendiente» y consultar después; un proceso de fondo puede requerir una respuesta inmediata para continuar. Hay que distinguir aceptación, ejecución y finalización, y expresar el significado de cada estado.

```
Respuesta que necesita el actor:

Resultado remoto ahora: evaluar petición-respuesta.
Aceptación duradera ahora: HTTP 202 y trabajo posterior.
Resultado más tarde: estado consultable y plazo observable.
```

- **Línea 1:** `Respuesta que necesita el actor:` → distingue el significado de la respuesta.
- **Línea 3:** `Resultado remoto ahora: evaluar petición-respuesta.` → el siguiente paso depende del resultado.
- **Línea 4:** `Aceptación duradera ahora: HTTP 202 y trabajo posterior.` → aceptar no equivale a completar.
- **Línea 5:** `Resultado más tarde: estado consultable y plazo observable.` → el retraso requiere una experiencia de usuario y una política de recuperación.

El criterio tiene dos consecuencias prácticas. La primera es que una interacción puede devolver aceptación inmediata y completar el trabajo después. La segunda es que un proceso de fondo también puede necesitar una respuesta HTTP para dar su siguiente paso. La combinación de ambas permite que el estilo se ajuste a la naturaleza de la operación.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La palabra «inmediata» requiere un plazo y un significado. Un actor puede necesitar un número de seguimiento en dos segundos y aceptar el resultado final después. Ese diseño permite una confirmación síncrona de aceptación y procesamiento asíncrono sin engañar al usuario.

Ejemplo concreto en AceriaApp. El flujo híbrido devuelve 202 cuando la planificación local y su outbox se han guardado. La respuesta no afirma que Inventario ya tenga la proyección. Un recurso consultable permite revisar el estado, y cada GET posterior sigue siendo una interacción de petición-respuesta.

Riesgo, alternativa y límite. No devuelvas 202 antes de conservar el trabajo de forma duradera. Si la única referencia queda en una cola en memoria y el proceso termina, el usuario recibió una aceptación que el sistema no puede cumplir. Tampoco uses 202 como sustituto de un error de validación conocido.

Comprobación técnica y aprendizaje. Fuerza un fallo de guardado local y comprueba que no se responde aceptación. En el caso correcto, consulta inmediatamente y luego tras el procesamiento para ver el estado eventual. El contrato de respuesta y la evidencia SQL deben contar la misma historia.

Bloque 3 — Criterios técnicos: tolerancia al fallo

El segundo criterio técnico es el efecto de un fallo y el retraso tolerable. Criticidad y sincronía son ejes distintos: una operación crítica puede requerir procesamiento asíncrono duradero, con recuperación, en vez de depender de una respuesta inmediata. Una operación que no puede continuar sin un dato debe fallar de forma explícita si ese dato no está disponible. Deben definirse pérdida aceptable, duplicación, tiempo de recuperación y resultado desconocido. Reintentos y compensaciones también pueden ser necesarios en procesos críticos.

```
Preguntas ante un fallo:

¿Se puede retrasar el trabajo?
¿Qué datos deben conservarse antes de aceptar?
¿Puede repetirse una solicitud o un evento?
¿Cómo se detecta y recupera un resultado desconocido?
```

- **Línea 1:** `Preguntas ante un fallo:` → evalúa el fallo sin asociar criticidad a un protocolo.
- **Línea 3:** `¿Se puede retrasar el trabajo?` → determina si el trabajo puede diferirse.
- **Línea 4:** `¿Qué datos deben conservarse antes de aceptar?` → fija el límite de aceptación duradera.
- **Línea 5:** `¿Puede repetirse una solicitud o un evento?` → exige idempotencia donde haya repetición.
- **Línea 6:** `¿Cómo se detecta y recupera un resultado desconocido?` → evita equiparar timeout con ausencia de efecto.

El criterio exige diseñar la recuperación de cada interacción. Los timeouts limitan esperas HTTP, pero no prueban que una escritura remota no haya ocurrido. Un outbox conserva una intención de integración tras un commit local, pero no completa por sí solo el trabajo remoto. La elección debe expresar qué se conserva, qué se reintenta y qué intervención necesita un fallo permanente.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Separa tolerancia a pérdida, tolerancia a retraso y tolerancia a duplicación. Un proceso crítico puede tolerar unos minutos de demora pero ninguna pérdida; otro necesita una respuesta instantánea y prefiere rechazar la operación cuando no puede comprobar un requisito.

Ejemplo concreto en AceriaApp. La proyección de planificación puede esperar mientras Inventario se recupera, siempre que Producción conserve el evento y se observe el retraso. La consulta informativa previa puede fallar y entonces el endpoint debe decidir explícitamente cómo responder; el outbox no repara una consulta que nunca produjo datos.

Riesgo, alternativa y límite. Un timeout no distingue siempre entre «el servidor no hizo nada» y «lo hizo pero perdí la respuesta». La idempotencia de comandos HTTP y la identidad de eventos resuelven problemas relacionados, pero se implementan en fronteras diferentes. No confundas la clave de una solicitud con un Guid nuevo en cada intento.

Comprobación técnica y aprendizaje. Para cada fallo de la matriz escribe el dato que sobrevive y el componente que reintenta. Comprueba que un rechazo técnico no aparece como un rechazo de negocio. Esa separación evita que una indisponibilidad temporal se convierta en una decisión incorrecta sobre existencias.

Bloque 4 — Criterios técnicos: consistencia requerida

El tercer criterio técnico es la invariante que debe mantenerse y la frontera que la protege. HTTP no convierte varias escrituras remotas en una transacción. Si una regla exige atomicidad estricta, primero se revisa si sus datos deben pertenecer al mismo propietario y a la misma transacción local. La coordinación distribuida tiene condiciones y costes específicos; no se obtiene añadiendo una llamada síncrona. La consistencia eventual permite propagar cambios después, pero requiere manejar retrasos, duplicados, orden y recuperación, por lo que tampoco es automáticamente más simple.

```
Consistencia requerida:

Invariante atómica: definir un propietario y una transacción.
Propagación diferida: observar retraso y convergencia.
Saga: transacciones locales y compensaciones.
HTTP o eventos: no sustituyen la frontera de consistencia.
```

- **Línea 1:** `Consistencia requerida:` → formula el requisito antes de elegir transporte.
- **Línea 3:** `Invariante atómica: definir un propietario y una transacción.` → busca una frontera que pueda proteger la regla.
- **Línea 4:** `Propagación diferida: observar retraso y convergencia.` → la proyección puede ir por detrás sin ser un error por sí sola.
- **Línea 5:** `Saga: transacciones locales y compensaciones.` → no promete atomicidad ACID global.
- **Línea 6:** `HTTP o eventos: no sustituyen la frontera de consistencia.` → ambos estilos requieren un diseño explícito de los datos.

El criterio tiene dos consecuencias prácticas. La primera es que una Saga coordina transacciones locales con compensaciones y estados intermedios; no proporciona aislamiento ni atomicidad ACID global. Two-Phase Commit es una coordinación transaccional diferente, con participantes compatibles y costes de disponibilidad. La segunda es que la consistencia eventual exige convergencia y recuperación; algunas operaciones necesitan compensación y otras solo una proyección idempotente. La combinación de ambas permite que el estilo se ajuste a los requisitos de consistencia.

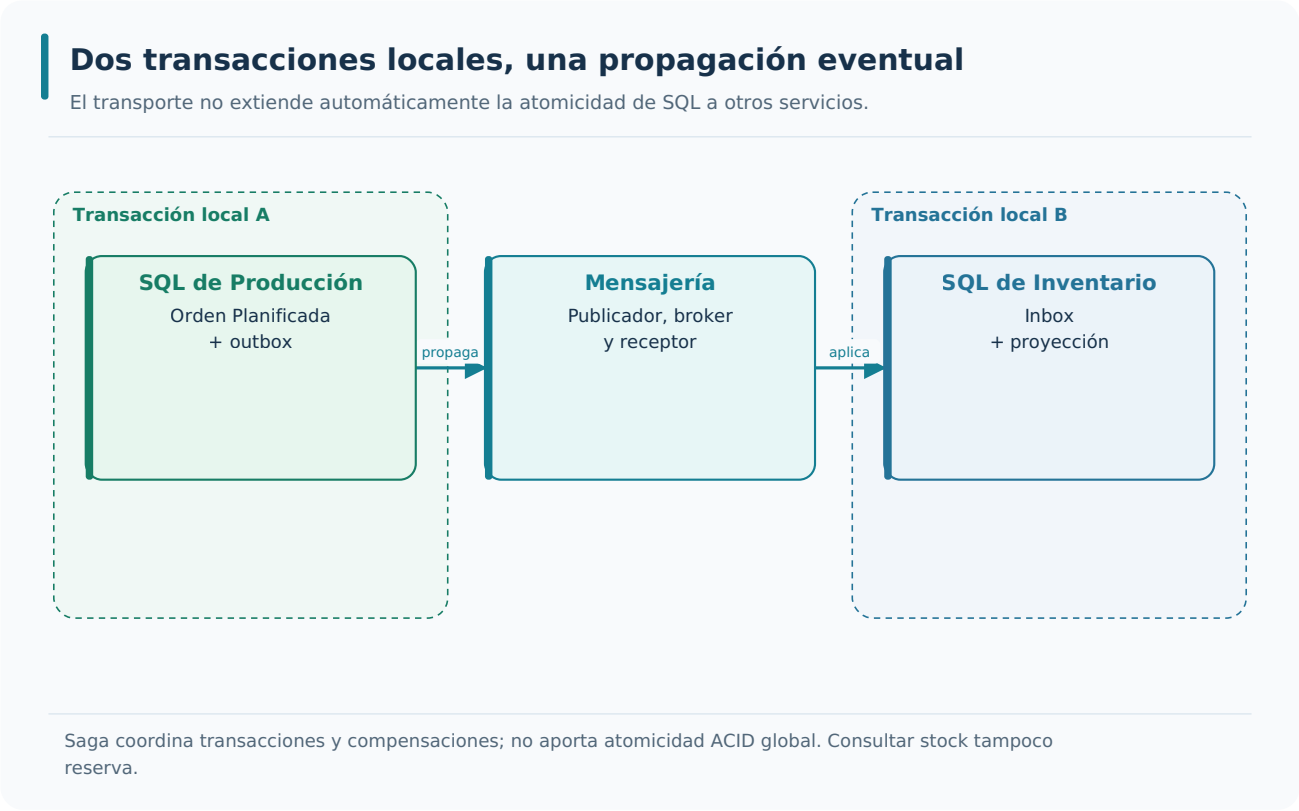
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Una invariante necesita un propietario capaz de hacerla cumplir en su frontera transaccional. Si no se permite que el stock disponible baje de cero, consultar desde Producción y guardar en otra base no protege esa regla. Inventario necesitaría una operación autoritativa de reserva o consumo.

Ejemplo concreto en AceriaApp. El módulo utiliza una regla más limitada: la transición de la orden y su evento pendiente se confirman juntos en Producción. En Inventario, inbox y proyección se confirman juntos. La atomicidad es local en cada caso y entre ambas bases hay una ventana eventual.

Riesgo, alternativa y límite. Una Saga puede modelar pasos, estados y compensaciones de una reserva futura. No convierte lecturas intermedias en aislamiento serializable ni hace que todos los participantes reviertan a la vez. Two-Phase Commit tampoco es una solución automática disponible por usar HTTP; requiere participantes y coordinación específicos.

Comprobación técnica y aprendizaje. Comprueba con una transacción fallida que no queda solo uno de los cambios locales. Después demuestra que ambos contextos pueden observar estados distintos durante una interrupción. Entender las dos observaciones permite describir con precisión qué garantiza el sistema y qué retrasa.



Interpretación del gráfico: Existen dos transacciones locales; el broker no convierte sus commits en una transacción global. La figura se distribuye en SVG vectorial y PNG.

Bloque 5 — Criterios técnicos: volumen y frecuencia

El cuarto criterio técnico es la relación entre tasa de llegada, capacidad de procesamiento, picos y latencia admitida. Una cola amortigua un pico si después hay capacidad para vaciarla; no elimina el trabajo ni produce capacidad infinita. HTTP puede atender alto volumen y la mensajería puede ser útil con bajo volumen. Ambos enfoques reutilizan conexiones. La frecuencia aislada no decide el protocolo: se miden tamaño de mensajes, concurrencia, latencias percentiles, retraso acumulado y coste operativo.

```
Capacidad y carga:

Tasa de llegada: mensajes o solicitudes por segundo.
Tasa de servicio: trabajo que puede completarse por segundo.
Si llegada supera servicio, aumenta la espera acumulada.
Medir percentiles de latencia, backlog y edad del pendiente.
Reutilizar conexiones en HTTP y en mensajería.
```

- **Línea 1:** `Capacidad y carga:` → describe la carga sin una receta binaria.
- **Línea 3:** `Tasa de llegada: mensajes o solicitudes por segundo.` → mide cuánto trabajo entra.
- **Línea 4:** `Tasa de servicio: trabajo que puede completarse por segundo.` → mide cuánto trabajo finaliza realmente.
- **Línea 5:** `Si llegada supera servicio, aumenta la espera acumulada.` → una cola amortigua picos, pero no crea capacidad.
- **Línea 6:** `Medir percentiles de latencia, backlog y edad del pendiente.` → hace visible la saturación y el retraso.

- **Línea 7:** `Reutilizar conexiones en HTTP y en mensajería.` → evita deducir el protocolo solo de la frecuencia.

El criterio tiene dos consecuencias prácticas. La primera es que las tasas y tamaños condicionan capacidad, almacenamiento y límites del broker. La segunda es que hay que evaluar la concurrencia y reutilización de conexiones en ambos estilos. La combinación de ambas permite que el estilo se ajuste a la carga del sistema.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La capacidad se razona con tasas y tiempo. Si entran cien eventos por segundo y salen ochenta, se acumulan veinte cada segundo mientras persista esa diferencia. Una cola ofrece un lugar para esperar, pero aumenta almacenamiento y latencia del resultado final.

Ejemplo concreto en AceriaApp. En Producción, la latencia del endpoint híbrido incluye consulta HTTP y commit local; la latencia de integración añade espera de outbox, broker y consumidor. Medir solo el tiempo del POST puede ocultar que Inventario tarda muchos minutos en reflejar la planificación.

Riesgo, alternativa y límite. La frecuencia no determina por sí sola la tecnología. HTTP usa conexiones persistentes y RabbitMQ mantiene conexiones de larga vida. Abrir una conexión por mensaje o por consulta desperdicia recursos, pero la solución es administrar su ciclo de vida, no elegir un protocolo a partir de una regla simplista.

Comprobación técnica y aprendizaje. Mide dos presupuestos: respuesta al actor y tiempo hasta la proyección. Prueba un pico que el sistema pueda drenar y otro que revele saturación. Define límites y alerta sobre la edad del pendiente, porque una cola pequeña con un mensaje muy antiguo también puede indicar un error permanente.

Bloque 6 — Criterios organizativos: autonomía de equipos

El primer criterio organizativo es la autonomía de los equipos. Los eventos pueden desacoplar despliegue y ritmo de procesamiento, pero sus contratos siguen requiriendo acuerdos de significado, versiones y retirada. Una API HTTP compatible también permite despliegue independiente. La autonomía se materializa cuando un cambio compatible puede publicarse sin detener a otros equipos, y cuando existen propietarios y procedimientos para resolver un cambio incompatible.

```
Autonomía de equipos:

Propietario claro de cada contrato y de sus datos.
Cambios compatibles y despliegues independientes.
Versiones y retirada acordadas para HTTP y eventos.
Mensajes antiguos pueden permanecer pendientes.
```

- **Línea 1:** `Autonomía de equipos:` → relaciona autonomía con acuerdos verificables.
- **Línea 3:** `Propietario claro de cada contrato y de sus datos.` → determina quién cambia y opera cada contrato.
- **Línea 4:** `Cambios compatibles y despliegues independientes.` → evita exigir despliegue simultáneo para cambios compatibles.
- **Línea 5:** `Versiones y retirada acordadas para HTTP y eventos.` → ambos transportes necesitan evolución controlada.
- **Línea 6:** `Mensajes antiguos pueden permanecer pendientes.` → el consumidor debe tolerar contratos emitidos antes de su despliegue.

El criterio tiene dos consecuencias prácticas. La primera es que la autonomía requiere contratos estables. La segunda es que incluso un equipo coordinado necesita compatibilidad durante un

despliegue gradual y con mensajes antiguos pendientes. La combinación de ambas permite que el estilo se ajuste a la organización del equipo.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La autonomía real depende de poder desplegar cambios compatibles, identificar propietarios y mantener contratos antiguos durante la transición. El transporte no elimina conversaciones entre equipos sobre significado, datos obligatorios o eliminación de campos.

Ejemplo concreto en AceriaApp. Producción puede añadir información opcional a una nueva evolución del evento si los consumidores existentes la toleran. Si cambia el significado de Cantidad o las unidades, no basta conservar el mismo nombre JSON: el contrato ha cambiado semánticamente y necesita una transición acordada.

Riesgo, alternativa y límite. Los mensajes pendientes hacen que convivan versiones antiguas con código nuevo. Una API HTTP también puede recibir solicitudes de clientes desactualizados. Retirar un contrato exige observar uso, comunicar un plazo y comprobar que los consumidores pueden migrar o reprocesar.

Comprobación técnica y aprendizaje. Guarda ejemplos representativos del contrato y prueba que el consumidor actual interpreta los mensajes admitidos. En el ADR identifica quién decide un cambio incompatible y quién opera la cola. Esa responsabilidad es más útil que etiquetar automáticamente «alta autonomía» como «asíncrona».

Bloque 7 — Criterios organizativos: madurez operativa

El segundo criterio organizativo es la madurez operativa: capacidad de monitorizar, diagnosticar y recuperar el recorrido completo. La mensajería añade colas, retención, duplicados y reproceso, y exige aprender a operarlos. HTTP puede simplificar un recorrido pequeño, pero también requiere métricas, límites y recuperación. La falta de experiencia no elimina un requisito de durabilidad: obliga a reducir alcance, formar al equipo o adoptar componentes administrados adecuados.

```
Capacidad operativa necesaria:

Detectar fallos y retrasos.
Relacionar solicitud, orden y evento.
Reintentar sin duplicar efectos.
Recuperar mensajes inválidos o pendientes.
Asignar responsables y comprobar restauración.
```

- **Línea 1:** `Capacidad operativa necesaria:` → convierte madurez en tareas observables.
- **Línea 3:** `Detectar fallos y retrasos.` → mide salud y tiempo pendiente, no solo procesos encendidos.
- **Línea 4:** `Relacionar solicitud, orden y evento.` → sigue el flujo entre componentes.
- **Línea 5:** `Reintentar sin duplicar efectos.` → la recuperación exige idempotencia.
- **Línea 6:** `Recuperar mensajes inválidos o pendientes.` → un error permanente necesita tratamiento definido.
- **Línea 7:** `Asignar responsables y comprobar restauración.` → la operación incluye personas y procedimientos.

El criterio tiene dos consecuencias prácticas. La primera es que la madurez operativa requiere inversión en herramientas. La segunda es que la falta de madurez requiere simplicidad. La combinación de ambas permite que el estilo se ajuste a la capacidad del equipo.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Operar mensajería significa saber qué hacer cuando el trabajo deja de avanzar. Tener un broker encendido no demuestra que las rutas existan, el publicador reciba confirmación o el consumidor pueda confirmar SQL. Cada tramo necesita señales observables.

Ejemplo concreto en AceriaApp. Un diagnóstico mínimo sigue OrdenId y EventoId, muestra pendientes de outbox y reconoce duplicados del inbox. Permite responder si la orden se guardó, si el broker aceptó el mensaje y si Inventario aplicó su proyección. Cada afirmación requiere su evidencia específica.

Riesgo, alternativa y límite. El recorrido envía errores de formato o versión a una cola de errores (DLQ) y reencola fallos técnicos. Esa distinción evita repetir indefinidamente un contrato imposible de interpretar, pero el requeue técnico inmediato puede crear bucles si el fallo persiste. Reintento diferido, alertas, retención, restauración y reproceso asistido requieren diseño adicional.

Comprobación técnica y aprendizaje. Realiza un ejercicio de recuperación con instrucciones reproducibles: detener consumidor, generar trabajo, observar pendiente y reanudar. Incluye un mensaje inválido en un entorno de prueba y documenta el resultado real. La capacidad del equipo se demuestra recuperando un escenario, no enumerando herramientas.

Bloque 8 — Patrones híbridos

Los patrones híbridos combinan estilos dentro de un flujo de negocio. En notificación con consulta posterior, un evento activa trabajo y una petición HTTP obtiene su estado cuando hace falta. En aceptación con finalización diferida, HTTP devuelve un identificador y `202 Accepted` después de registrar de forma duradera el trabajo; un consumidor lo procesa después. El sondeo posterior mediante GET sigue siendo comunicación HTTP de petición-respuesta, aunque se ejecute periódicamente. Otra combinación consulta información por HTTP, confirma una transición local y propaga el hecho mediante outbox. Event Sourcing es un patrón de persistencia que reconstruye estado desde eventos; no es por definición un patrón de comunicación híbrida ni se implementa en este módulo.

```
Flujo híbrido del curso:

Consultar stock informativo por HTTP.
Planificar la orden y guardar outbox en SQL.
Responder 202 con referencia a la orden.
Publicar y consumir en segundo plano.
Consultar la proyección, que puede aparecer después.
```

- **Línea 1:** `Flujo híbrido del curso:` → combina interacción inmediata y propagación posterior.
- **Línea 3:** `Consultar stock informativo por HTTP.` → obtiene una observación de existencias, sin reservar.
- **Línea 4:** `Planificar la orden y guardar outbox en SQL.` → confirma solo el estado y la intención de integración locales.
- **Línea 5:** `Responder 202 con referencia a la orden.` → el contrato comunica que la propagación continúa pendiente.
- **Línea 6:** `Publicar y consumir en segundo plano.` → los trabajadores conservan y deduplican la entrega.
- **Línea 7:** `Consultar la proyección, que puede aparecer después.` → la consulta HTTP posterior puede no ver todavía el evento.

Los patrones híbridos tienen dos consecuencias prácticas. La primera es que combinan las ventajas de ambos estilos. La segunda es que añaden complejidad a la implementación. La combinación de ambas permite que el sistema se adapte a interacciones complejas.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Un flujo híbrido debe conservar el significado de cada etapa. La consulta de stock produce una observación. El commit local produce una orden planificada y un evento pendiente. La publicación produce aceptación por el broker. El consumo produce una proyección de Inventario.

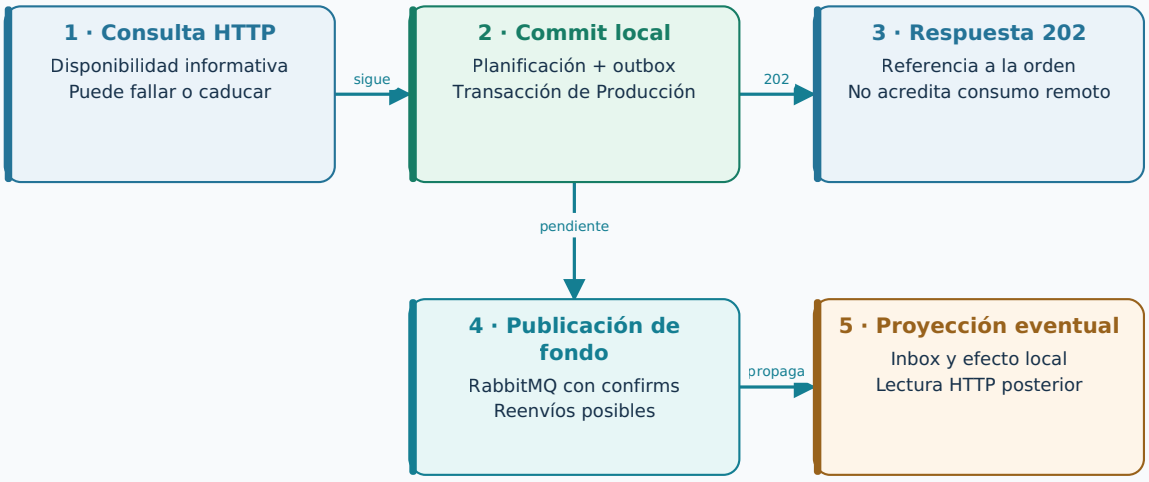
Ejemplo concreto en AceriaApp. El usuario recibe 202 después del commit local, antes de que el flujo remoto deba terminar. La vista eventual puede consultarse más tarde. Si el consumidor estuvo detenido, la planificación local sigue existiendo y la lectura remota puede mostrar todavía ausencia de proyección.

Riesgo, alternativa y límite. No mantengas una transacción SQL abierta mientras esperas a que Inventario consuma el evento: reintroduciría dependencia temporal y bloquearía recursos sin aportar una transacción global. Tampoco atribuyas Event Sourcing al uso de outbox; el estado autoritativo sigue guardado como estado actual del agregado.

Comprobación técnica y aprendizaje. Verifica el orden de los hitos con marcas y consultas. Una prueba debe poder observar el estado local antes del remoto sin considerarlo fallo, y comprobar convergencia cuando el consumidor vuelve. La ausencia permanente o un error no recuperado sí exige diagnóstico.

Flujo híbrido: aceptación local y resultado diferido

El endpoint de planificación combina una observación HTTP con un hecho que se propaga.



202 se emite tras guardar el trabajo. El flujo no implementa reserva, descuento de stock, facturación ni Event Sourcing.

Interpretación del gráfico: 202 confirma aceptación local tras guardar. La proyección se actualiza después y no representa una reserva de material. La figura se distribuye en SVG vectorial y PNG.

Bloque 9 — Matriz de decisión

La matriz de decisión resume los criterios en una tabla de doble entrada. Las filas representan los criterios. Las columnas representan los estilos de comunicación. Cada celda indica la recomendación

del criterio para el estilo correspondiente. La matriz se usa como guía en el momento de decidir. La matriz se documenta en el ADR de la decisión.

```
Registro de decisión por interacción:

Necesidad del actor y plazo esperado.
Datos que confirma la respuesta.
Frontera transaccional e identidad de reintento.
Política de fallo, duplicados y recuperación.
Métricas y criterio para revisar la decisión.
```

- **Línea 1:** `Registro de decisión por interacción:` → resume lo que debe justificar el ADR.
- **Línea 3:** `Necesidad del actor y plazo esperado.` → define la experiencia del actor.
- **Línea 4:** `Datos que confirma la respuesta.` → aporta semántica concreta al resultado.
- **Línea 5:** `Frontera transaccional e identidad de reintento.` → delimita atomicidad y repetición.
- **Línea 6:** `Política de fallo, duplicados y recuperación.` → explica cómo se mantiene la corrección ante fallos.
- **Línea 7:** `Métricas y criterio para revisar la decisión.` → hace que la decisión sea revisable con evidencia.

La matriz tiene dos consecuencias prácticas. La primera es que la decisión se basa en criterios explícitos. La segunda es que la decisión se documenta y se revisa. La combinación de ambas permite que la elección sea razonada y trazable.

Criterio	Petición-respuesta HTTP	Eventos y procesamiento diferido
Resultado necesario ahora	Natural si el destino responde dentro del presupuesto	Puede requerir aceptación y consulta posterior
Interrupción del destino	La operación dependiente necesita rechazo o alternativa explícita	Puede acumular trabajo si se conserva y existe capacidad
Consistencia	No aporta una transacción global	Requiere convergencia y manejo del retraso
Volumen y picos	Limitar concurrencia y medir latencia	Amortiguar picos y medir edad del pendiente
Frecuencia	Reutilizar conexiones	Reutilizar conexiones; no decide por sí sola
Autonomía	Contratos compatibles y propietarios claros	Contratos compatibles, versiones y consumidores pendientes
Madurez operativa	Diagnóstico HTTP, límites y recuperación	Lo anterior más topología, reentrega y reproceso
Criticidad	Diseñar aceptación, idempotencia y resultado incierto	Diseñar conservación, idempotencia y recuperación

Ampliación didáctica aplicada a AceriaApp

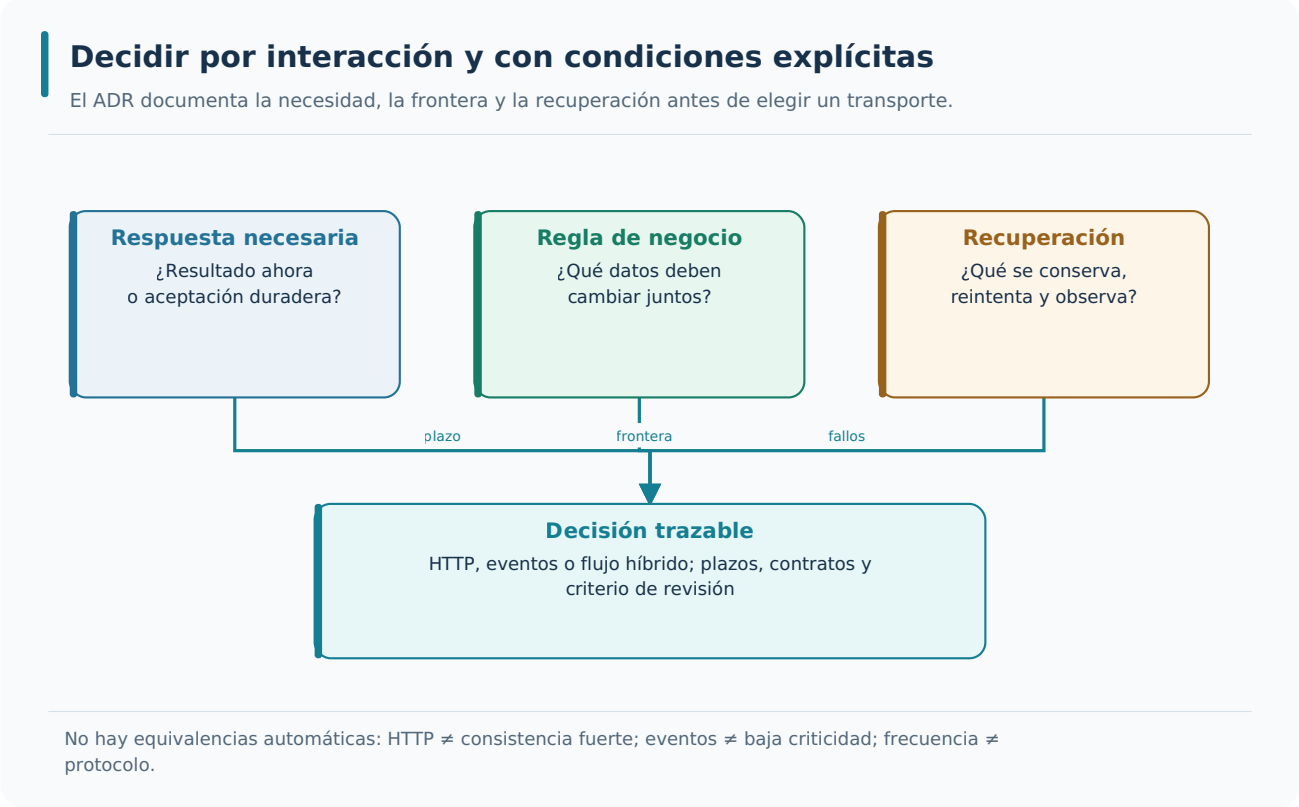
La decisión arquitectónica. La matriz sirve para organizar preguntas y consecuencias, no para sumar votos mecánicamente. Un requisito obligatorio de atomicidad o de plazo puede descartar una opción aunque otros criterios parezcan favorecerla. Debe quedar claro cuál es la restricción dominante.

Ejemplo concreto en AceriaApp. Para la disponibilidad informativa, importa responder pronto con un dato o un error explícito. Para la proyección de planificación, importa conservar el hecho y

recuperarlo aunque Inventario esté detenido. Para una reserva garantizada futura, habría que definir otra operación y otra frontera de consistencia.

Riesgo, alternativa y límite. No escribas «síncrona = fuerte» o «asíncrona = no crítica». Tampoco «alta frecuencia = HTTP». Esas equivalencias mezclan objetivos, transporte y propiedades que solo aparecen bajo un diseño concreto. La tabla corregida explica ventajas y costes sin prometer resultados automáticos.

Comprobación técnica y aprendizaje. Aplica la matriz a un caso nuevo y redacta un ADR breve. Otra persona debería poder cuestionar la decisión cambiando una premisa, como el plazo admisible o la pérdida tolerable. Si la conclusión no cambia ante ningún requisito, probablemente se eligió la tecnología antes de razonar.



Interpretación del gráfico: La elección depende de respuesta, invariante y recuperación por interacción; no de una clasificación binaria del sistema. La figura se distribuye en SVG vectorial y PNG.

Bloque 10 — Cierre del módulo 6

El módulo ha presentado HTTP, clientes tipados, eventos, contratos de integración y criterios técnicos y organizativos. La aplicación conserva el dominio y el estado de Producción y añade un servicio de Inventario separado. La validación abarca puertos, manejo de errores, SQL Server, outbox, RabbitMQ, inbox y consulta de la proyección, según la evidencia ejecutada en las prácticas.

El cierre prepara observabilidad y resiliencia: seguimiento de una solicitud, correlación de una orden y evento, retraso pendiente y recuperación. El sistema no se considera completo por añadir un broker; debe poder explicar qué confirmó cada componente y qué queda pendiente.

```
Cierre del módulo 6:

HTTP: cliente tipado, errores diferenciados y cancelación.
Eventos: outbox, broker, confirms, consumidor e inbox.
Híbrido: consulta informativa, planificación y proyección eventual.
Decisión: criterios técnicos, organizativos y ADR.
Evidencia: compilación, pruebas, SQL y recorrido HTTP entre procesos.
```

- **Línea 1:** `Cierre del módulo 6:` → relaciona aprendizaje y comprobación.
- **Línea 3:** `HTTP: cliente tipado, errores diferenciados y cancelación.` → resume la comunicación inmediata.
- **Línea 4:** `Eventos: outbox, broker, confirms, consumidor e inbox.` → resume la propagación duradera y deduplicada.
- **Línea 5:** `Híbrido: consulta informativa, planificación y proyección eventual.` → declara el flujo real y sus límites.
- **Línea 6:** `Decisión: criterios técnicos, organizativos y ADR.` → documenta por qué cada interacción usa un estilo.
- **Línea 7:** `Evidencia: compilación, pruebas, SQL y recorrido HTTP entre procesos.` → distingue la ejecución real de un catálogo de conceptos.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El cierre integra lo aprendido sin ampliar silenciosamente el alcance. Producción conserva su modelo DDD y su acceso SQL; Inventario aporta un servicio separado con consulta HTTP y proyección de planificación. Los puertos mantienen las decisiones del caso de uso fuera de los detalles de transporte.

Ejemplo concreto en AceriaApp. La evidencia debe cubrir compilación, pruebas útiles y ejecución entre procesos con SQL Server y RabbitMQ. El catálogo de decisiones y los ADR ayudan a comprender el diseño, mientras los resultados HTTP, filas SQL y observación de mensajes demuestran que el recorrido ocurrió.

Riesgo, alternativa y límite. Quedan fuera la reserva garantizada, el descuento de stock, Facturacion.Api y Event Sourcing. Tampoco se promete tolerancia ilimitada a fallos: los reintentos necesitan datos conservados, capacidad y componentes que recuperen servicio. Estas fronteras permiten ampliar el sistema con honestidad técnica.

Comprobación técnica y aprendizaje. Revisa un caso satisfactorio, un error HTTP remoto, una interrupción del broker, una reentrega duplicada y una lectura temporalmente atrasada según las pruebas disponibles. Registra qué se ejecutó y qué quedó pendiente. Esa revisión prepara observabilidad y resiliencia sin confundir el aprendizaje con una garantía industrial universal.

Resumen rápido de la teoría

- La decisión se toma por interacción con necesidades, plazos y fallos explícitos.
- Criticidad, consistencia y transporte son dimensiones diferentes.
- Una Saga no proporciona atomicidad ACID global y Event Sourcing no es un patrón híbrido por definición.
- Volumen y frecuencia se analizan con tasas, capacidad, latencia y retraso acumulado.
- HTTP y eventos requieren contratos compatibles y capacidad de operación.
- El flujo híbrido confirma localmente y devuelve 202 antes de completar la proyección remota.
- El ADR y las pruebas registran decisiones y evidencia diferentes.

Mapa del contrato HTTP del recorrido

Proceso	Método y ruta	Significado
Inventario.Api	`GET /api/v1/materiales/{material}`	Consulta el material; 404 se distingue del fallo técnico
API de Producción	`GET /api/v1/comunicacion/materiales/{material}`	Demuestra el cliente HTTP tipado desde 6.1
API de Producción	`POST /api/v1/comunicacion/ordenes/{id}/planificar`	Desde 6.2 confirma planificación y outbox, devuelve 202; en 6.3 añade consulta informativa previa
Inventario.Api	`GET /api/v1/planificaciones/{ordenId}`	Lee la proyección eventual de una planificación

Los códigos concretos de rechazo y los cuerpos completos se comprueban en las prácticas. Un 202 confirma aceptación local y no acredita consumo remoto ni reserva de existencias.

Bibliografía oficial y lecturas de ampliación

- Microsoft Learn: IHttpClientFactory. Registro de clientes tipados y administración de manejadores.
- Microsoft Learn: directrices para HttpClient. Ciclo de vida de conexiones y clientes.
- RFC 9110: HTTP Semantics. Métodos, códigos de estado y significado de 202 Accepted.
- RFC 9114: HTTP/3. Transporte QUIC y semántica de HTTP/3.
- RabbitMQ: API .NET/C#. IChannel, operaciones asíncronas, concurrencia y seguridad de la memoria del consumidor.
- RabbitMQ: publisher confirms. Configuración y espera de confirmaciones en el cliente .NET.
- RabbitMQ: acknowledgements y confirms. Alcance diferente de las confirmaciones del publicador y del consumidor.
- RabbitMQ: reliability guide. Pérdidas de conexión, reenvíos, duplicados y responsabilidades de aplicación.
- RabbitMQ .NET Client: migración a v7. Cambios de API y validez del cuerpo durante el callback.

Las referencias amplían los mecanismos. El contrato y el alcance del laboratorio se verifican con los archivos y resultados de las prácticas del módulo.